

## **INTRODUCTION TO ARDUINO**

### **Arduino Uno**

The Arduino Uno is an open-source microcontroller board based on the Microchip ATmega328P microcontroller and developed by Arduino.cc.

Along with ATmega328P, it consists other components such as crystal oscillator, serial communication, voltage regulator etc to support the microcontroller, a USB connection, a power barrel jack, an ICSP header and a reset button.

The board is equipped with sets of digital and analog input/output (I/O) pins that may be interfaced to various expansion boards (shields) and other circuits .

The board has 14 digital I/O pins (six capable of PWM output), 6 analog I/O pins, and is programmable with the Arduino IDE (Integrated Development Environment), via a type B USB cable.

It can be powered by the USB cable or by an external 9-volt battery, though it accepts voltages between 7 and 20 volts.



Figure: Arduino UNO Board

## Arduino Uno Technical Specifications

|                           |                                               |
|---------------------------|-----------------------------------------------|
| Microcontroller           | ATmega328P – 8 bit AVR family microcontroller |
| Operating Voltage         | 5V                                            |
| Recommended Input Voltage | 7-12V                                         |
| Input Voltage Limits      | 6-20V                                         |
| Analog Input Pins         | 6 (A0 – A5)                                   |
| Digital I/O Pins          | 14 (Out of which 6 provide PWM output)        |
| DC Current on I/O Pins    | 40 mA                                         |
| DC Current on 3.3V Pin    | 50 mA                                         |
| Flash Memory              | 32 KB (0.5 KB is used for Bootloader)         |
| SRAM                      | 2 KB                                          |
| EEPROM                    | 1 KB                                          |

## Pin Description

| Pin Category        | Pin Name            | Details                                                                                                                                                                                                                                                                                                        |
|---------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Power               | Vin, 3.3V, 5V, GND  | <b>Vin:</b> Input voltage to Arduino when using an external power source.<br><b>3.3V:</b> 3.3V supply generated by on-board voltage regulator. Maximum current draw is 50mA.<br><b>5V:</b> Regulated power supply used to power microcontroller and other components on the board.<br><b>GND:</b> ground pins. |
| Reset               | Reset               | Resets the microcontroller.                                                                                                                                                                                                                                                                                    |
| Analog Pins         | A0 – A5             | Used to provide analog input in the range of 0-5V                                                                                                                                                                                                                                                              |
| Input/Output Pins   | Digital Pins 0 - 13 | Can be used as input or output pins.                                                                                                                                                                                                                                                                           |
| Serial              | 0(Rx), 1(Tx)        | Used to receive and transmit TTL serial data.                                                                                                                                                                                                                                                                  |
| External Interrupts | 2, 3                | To trigger an interrupt.                                                                                                                                                                                                                                                                                       |

|             |                                            |                                                 |
|-------------|--------------------------------------------|-------------------------------------------------|
| PWM         | 3, 5, 6, 9, 11                             | Provides 8-bit PWM output.                      |
| SPI         | 10 (SS), 11 (MOSI), 12 (MISO) and 13 (SCK) | Used for SPI communication.                     |
| Inbuilt LED | 13                                         | To turn on the inbuilt LED.                     |
| TWI         | A4 (SDA), A5 (SCA)                         | Used for TWI communication.                     |
| AREF        | AREF                                       | To provide reference voltage for input voltage. |

## How to use Arduino Board

The 14 digital input/output pins can be used as input or output pins by using `pinMode()`, `digitalRead()` and `digitalWrite()` functions in arduino programming. Each pin operate at 5V and can provide or receive a maximum of 40mA current, and has an internal pull-up resistor of 20-50KOhms which are disconnected by default. Out of these 14 pins, some pins have specific functions as listed below:

- Serial Pins 0 (Rx) and 1 (Tx): Rx and Tx pins are used to receive and transmit TTL serialdata. They are connected with the corresponding ATmega328P USB to TTL serial chip.
- External Interrupt Pins 2 and 3: These pins can be configured to trigger an interrupt on a low value, a rising or falling edge, or a change in value.
- PWM Pins 3, 5, 6, 9 and 11: These pins provide an 8-bit PWM output by using `analogWrite()` function.
- SPI Pins 10 (SS), 11 (MOSI), 12 (MISO) and 13 (SCK): These pins are used for SPI communication.
- In-built LED Pin 13: This pin is connected with an built-in LED, when pin 13 is HIGH – LED is on and when pin 13 is LOW, its off.

Along with 14 Digital pins, there are 6 analog input pins, each of which provide 10 bits of resolution, i.e. 1024 different values. They measure from 0 to 5 volts but this limit can be increased by using AREF pin with `analogReference()` function.

- Analog pin 4 (SDA) and pin 5 (SCA) also used for TWI communication using Wire library.

Arduino Uno has a couple of other pins as explained below:

- AREF: Used to provide reference voltage for analog inputs with analogReference() function.
- Reset Pin: Making this pin LOW, resets the microcontroller.

### **Communication**

Arduino can be used to communicate with a computer, another Arduino board or other microcontrollers. The ATmega328P microcontroller provides UART TTL (5V) serial communication which can be done using digital pin 0 (Rx) and digital pin 1 (Tx). An ATmega16U2 on the board channels this serial communication over USB and appears as a virtual com port to software on the computer. The ATmega16U2 firmware uses the standard USB COM drivers, and no external driver is needed. However, on Windows, a .inf file is required. The Arduino software includes a serial monitor which allows simple textual data to be sent to and from the Arduino board. There are two RX and TX LEDs on the arduino board which will flash when data is being transmitted via the USB-to-serial chip and USB connection to the computer (not for serial communication on pins 0 and 1). A Software Serial library allows for serial communication on any of the Uno's digital pins. The ATmega328P also supports I2C (TWI) and SPI communication. The Arduino software includes a Wire library to simplify use of the I2C bus.

## Arduino Uno to ATmega328 Pin Mapping

| Arduino function    |                          |    |    |                        |  | Arduino function     |
|---------------------|--------------------------|----|----|------------------------|--|----------------------|
| reset               | (PCINT14/RESET) PC6      | 1  | 28 | PC5 (ADC5/SCL/PCINT13) |  | analog input 5       |
| digital pin 0 (RX)  | (PCINT16/RXD) PD0        | 2  | 27 | PC4 (ADC4/SDA/PCINT12) |  | analog input 4       |
| digital pin 1 (TX)  | (PCINT17/TXD) PD1        | 3  | 26 | PC3 (ADC3/PCINT11)     |  | analog input 3       |
| digital pin 2       | (PCINT18/INT0) PD2       | 4  | 25 | PC2 (ADC2/PCINT10)     |  | analog input 2       |
| digital pin 3 (PWM) | (PCINT19/OC2B/INT1) PD3  | 5  | 24 | PC1 (ADC1/PCINT9)      |  | analog input 1       |
| digital pin 4       | (PCINT20/XCK/T0) PD4     | 6  | 23 | PC0 (ADC0/PCINT8)      |  | analog input 0       |
| VCC                 | VCC                      | 7  | 22 | GND                    |  | GND                  |
| GND                 | GND                      | 8  | 21 | AREF                   |  | analog reference     |
| crystal             | (PCINT6/XTAL1/TOSC1) PB6 | 9  | 20 | AVCC                   |  | VCC                  |
| crystal             | (PCINT7/XTAL2/TOSC2) PB7 | 10 | 19 | PB5 (SCK/PCINT5)       |  | digital pin 13       |
| digital pin 5 (PWM) | (PCINT21/OC0B/T1) PD5    | 11 | 18 | PB4 (MISO/PCINT4)      |  | digital pin 12       |
| digital pin 6 (PWM) | (PCINT22/OC0A/AIN0) PD6  | 12 | 17 | PB3 (MOSI/OC2A/PCINT3) |  | digital pin 11(PWM)  |
| digital pin 7       | (PCINT23/AIN1) PD7       | 13 | 16 | PB2 (SS/OC1B/PCINT2)   |  | digital pin 10 (PWM) |
| digital pin 8       | (PCINT0/CLKO/ICP1) PB0   | 14 | 15 | PB1 (OC1A/PCINT1)      |  | digital pin 9 (PWM)  |

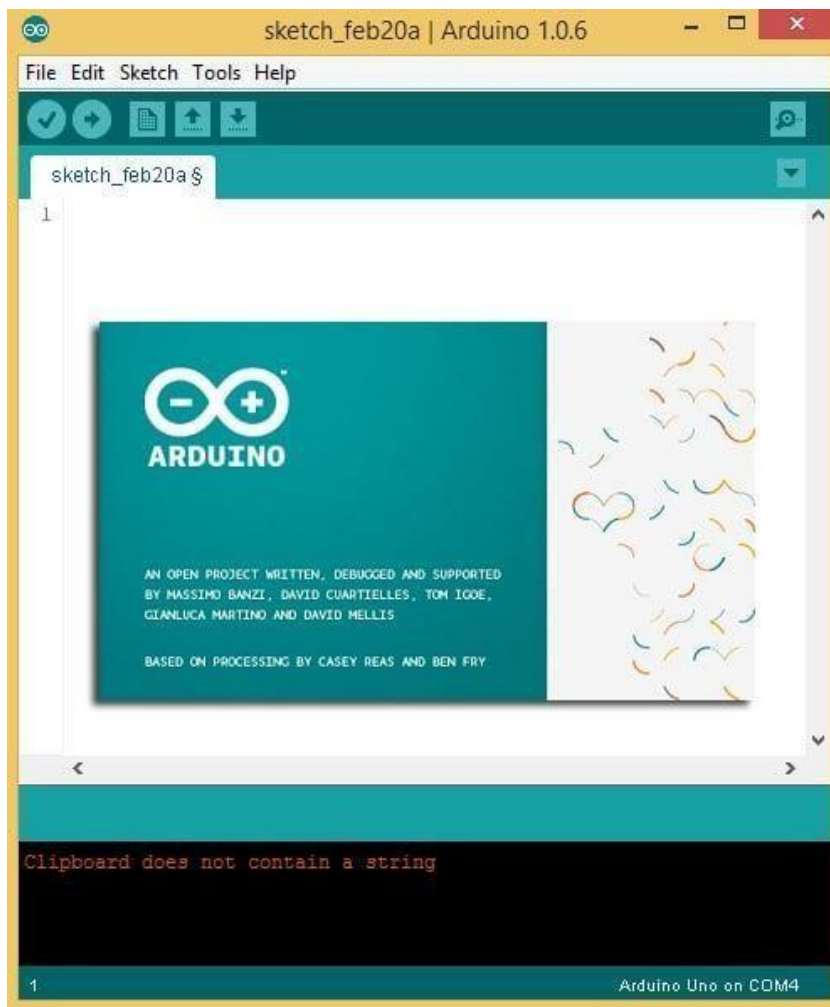
Digital Pins 11, 12 & 13 are used by the ICSP header for MOSI, MISO, SCK connections (Atmega168 pins 17, 18 & 19). Avoid low-impedance loads on these pins when using the ICSP header.

## Software

Arduino IDE (Integrated Development Environment) is required to program the Arduino Unoboard.

### Installation steps of Arduino IDE

#### Getting Started with the Arduino – Installing the IDE



So you just got an Arduino! They're definitely a lot of fun, but what's next? Where do you start? Well, to do anything useful with it, you're going to need to upload some code to it. To do that, you'll first need to install the Arduino IDE (Integrated Development Environment) on another computer.

The IDE compiles (converts) the code you write into instructions the Arduino can understand. The code is typed (or copy/pasted) into the IDE, and sent to the Arduino via a USB cable.

The Arduino IDE software is free to download, and installing it is really easy. Just follow the instructions below and you'll be writing code and getting your Arduino up and running in no time.

## **INSTALLING THE ARDUINO IDE**

1. Visit <http://www.arduino.cc/en/main/software> to download the latest Arduino IDE version for your computer's operating system. There are versions for Windows, Mac, and Linux systems. At the download page, click on the "Windows Installer" option for the easiest installation.
2. Save the .exe file to your hard drive.
3. Open the .exe file.
4. Click the button to agree to the licensing agreement:



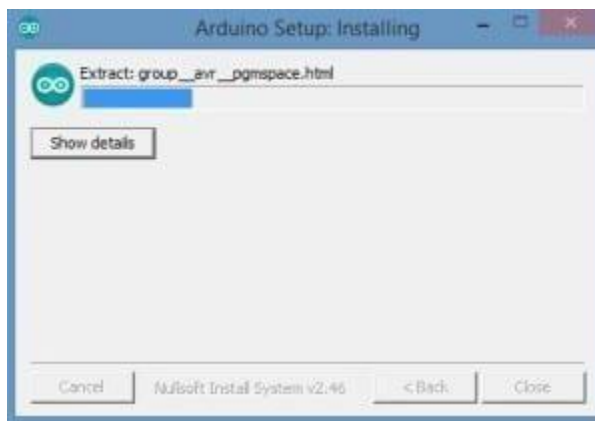
5. Decide which components to install, then click "Next":



6. Select which folder to install the program to, then click “Install”:



7. Wait for the program to finish installing, then click “Close”:



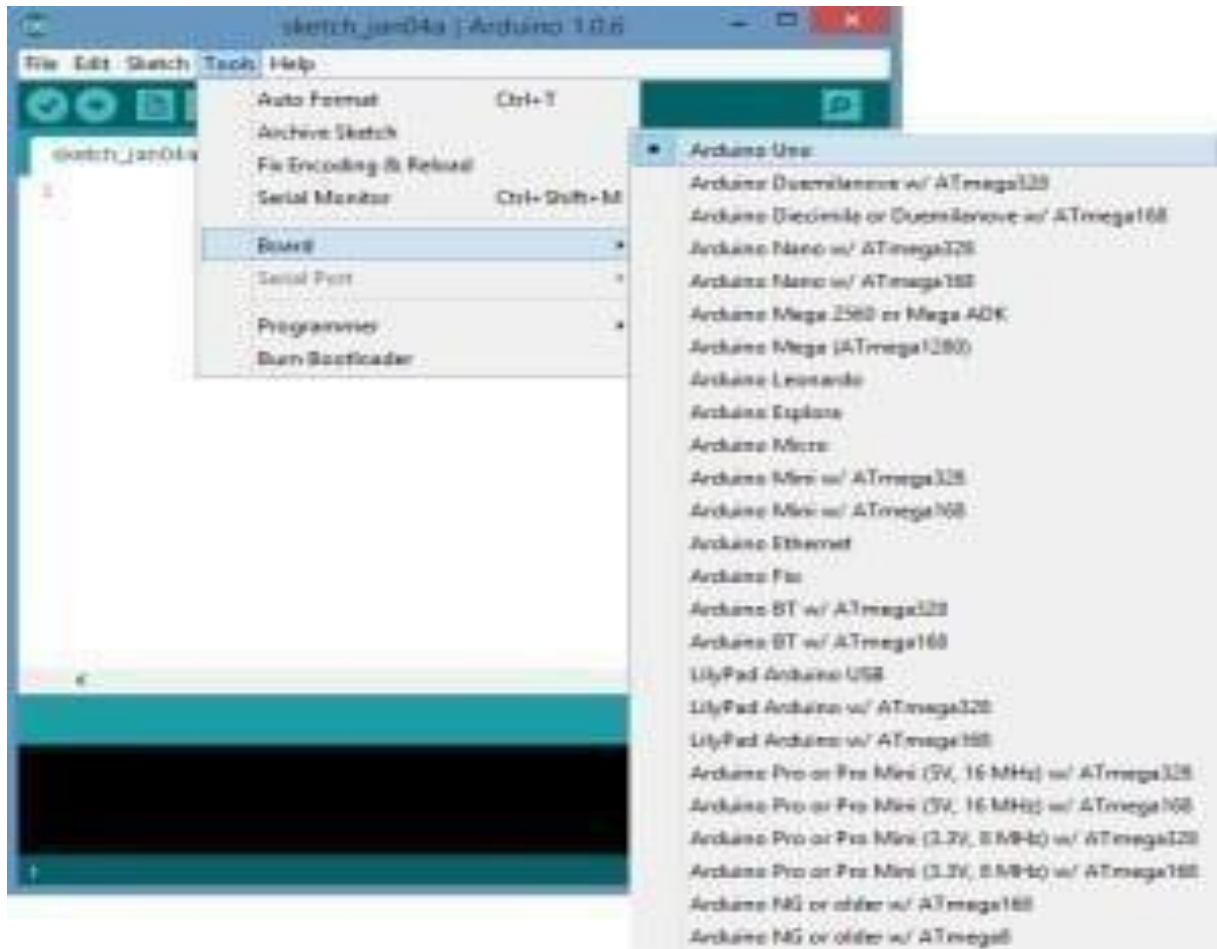
8. Now find the Arduino shortcut on your Desktop and click on it. The IDE will open up and you’ll see the code editor:





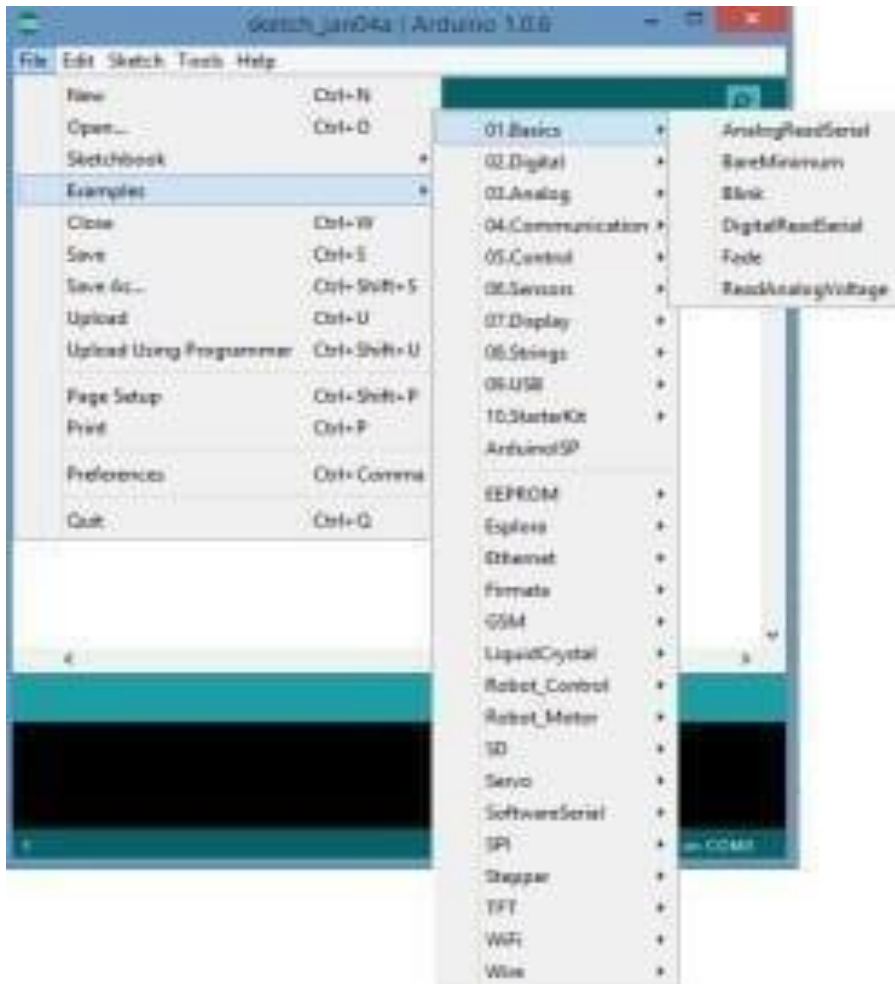
## CONFIGURING THE ARDUINO IDE

The next thing to do is to make sure the software is set up for your particular Arduino board. Goto the “Tools” drop-down menu, and find “Board”. Another menu will appear, where you can select from a list of Arduino models. I have the Arduino Uno R3, so I chose “Arduino Uno”.



## EXPLORING THE ARDUINO IDE

If you want, take a minute to browse through the different menus in the IDE. There is a good variety of example programs that come with the IDE in the “Examples” menu. These will help you get started with your Arduino right away without having to do lots of research.



## Introduction to Raspberry Pi

### What is Raspberry Pi?

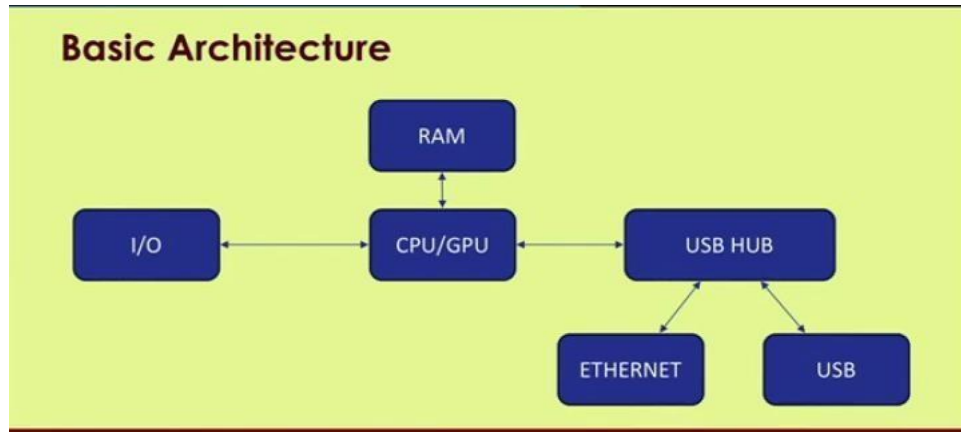
It is basically micro sized computer or commonly in common terms it is said as a computer in your palm more specifically it is a single board computer which is very low cost device and which is very easy to access.

- Raspberry pi compared to Arduino is more powerful, it is more powerful in terms of the computation or processing power.
- Additionally it has better memory capacity and also it can integrate different types of sensors and actuators and this part is more attractive than compared to the similar kind of feature of Arduino.
- We can have different types of sensors integration in Raspberry pi and due to the feature that it can process more compared to Arduino it has better processing capabilities and more features and so on.

| Specifications |                        |                        |                       |
|----------------|------------------------|------------------------|-----------------------|
| Key features   | Raspberry pi 3 model B | Raspberry pi 2 model B | Raspberry Pi zero     |
| RAM            | 1GB SDRAM              | 1GB SDRAM              | 512 MB SDRAM          |
| CPU            | Quad cortex A53@1.2GHz | Quad cortex A53@900MHz | ARM 11@ 1GHz          |
| GPU            | 400 MHz video core IV  | 250 MHz video core IV  | 250 MHz video core IV |
| Ethernet       | 10/100                 | 10/100                 | None                  |
| Wireless       | 802.11/Bluetooth 4.0   | None                   | None                  |
| Video output   | HDMI/Composite         | HDMI/Composite         | HDMI/Composite        |
| GPIO           | 40                     | 40                     | 40                    |

- There is a provision for Wi-Fi and Bluetooth only on mod pi 3.  
Generally video output is from HDMI port and there are 40 GPIO pins. So, these GPIO pins are mainly known as general purpose input output pins.

| S.No. | AURDUINO                                                                                                 | RASPBERRY PI                                                                                   |  |
|-------|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|--|
| 1.    | Arduino is a microcontroller, which is a part of the computer. It runs only one program again and again. | It is a mini computer with Raspbian OS. It can run multiple programs at a time.                |  |
| 2.    | Arduino requires external hardware to connect to the internet                                            | Raspberry Pi can be easily connected to the internet using Ethernet port and USB Wi-Fi dongles |  |
| 3.    | Arduino has only one USB port to connect to the computer.                                                | Raspberry Pi has 4 USB ports to connect different devices.                                     |  |
| 4.    | Processor used in Arduino is from AVR family Atmega328P                                                  | The processor used is from ARM family.                                                         |  |
| 5.    | Arduino uses Arduino, C/C++ type of coding                                                               | The Recommended programming language is python but C, C++, Python, ruby are preferred          |  |
| 6.    | Arduino can provide onboard storage but limited memory.                                                  | Raspberry Pi did not have storage on board. It provides an SD card port.                       |  |
| 7.    | NO inbuilt wifi, bluetooth facility                                                                      | Inbuilt wifi, bluetooth facility                                                               |  |
| 8.    | No on board port for camera                                                                              | onboard port for camera                                                                        |  |
| 9     | Low Cost                                                                                                 | Relatively high cost                                                                           |  |



This is the basic functional architecture of a raspberry pi. The center you have a CPU or GPU you have various input output ports connected to it you have a RAM you have a USB hub from which you can connect an Ethernet as well as you have various USB ports to which you can connect regular USB devices.

### **Raspberry Pi3 b+**

- Raspberry Pi is a small single board computer. By connecting peripherals like Keyboard, mouse, display to the Raspberry Pi, it will act as a mini personal computer.
- Raspberry Pi is popularly used for real time Image/Video Processing; IoT based applications and Robotics applications.
- Raspberry Pi is slower than laptop or desktop but is still a computer which can provide all the expected features or abilities, at low power consumption.
- Raspberry Pi Foundation officially provides Debian based Raspbian OS. Also, they provide NOOBS OS for Raspberry Pi. We can install several Third-Party versions of OS like Ubuntu, Archlinux, RISC OS, Windows 10 IOT Core, etc.
- Raspbian OS is official Operating System available for free to use. This OS is efficiently optimized to use with Raspberry Pi.
- Raspbian have GUI which includes tools for Browsing, Python programming, office, games, etc.
- We should use SD card (minimum 16 GB recommended) to store the OS (operating System).

- Raspberry Pi is more than computer as it provides access to the on-chip hardware i.e. GPIOs for developing an application. By accessing GPIO, we can connect devices like LED, motors, sensors, etc and can control them too.
- It has ARM based Broadcom Processor SoC along with on-chip GPU (Graphics Processing Unit).
- The CPU speed of Raspberry Pi varies from 700 MHz to 1.2 GHz. Also, it has on-board SDRAM that ranges from 256 MB to 1 GB.
- Raspberry Pi also provides on-chip SPI, I2C, and UART modules.

There are different versions of raspberry pi available as listed below:

- Raspberry Pi 1 Model A
- Raspberry Pi 1 Model A+
- Raspberry Pi 1 Model B
- Raspberry Pi 1 Model B+
- Raspberry Pi 2 Model B
- Raspberry Pi 3 Model B
- Raspberry Pi Zero

Out of the above versions of Raspberry Pi, more prominently use Raspberry Pi and their features are as follows:

| <b>Features</b>        | <b>Raspberry Pi Model B+</b> | <b>Raspberry Pi 2 Model B</b> | <b>Raspberry Pi 3 Model B</b> | <b>Raspberry Pi zero</b> |
|------------------------|------------------------------|-------------------------------|-------------------------------|--------------------------|
| <b>SoC</b>             | BCM2835                      | BCM2836                       | BCM2837                       | BCM2835                  |
| <b>CPU</b>             | ARM11                        | Quad Cortex A7                | Quad Cortex A53               | ARM11                    |
| <b>Operating Freq.</b> | 700 MHz                      | 900 MHz                       | 1.2 GHz                       | 1 GHz                    |
| <b>RAM</b>             | 512 MB SDRAM                 | 1 GB SDRAM                    | 1 GB SDRAM                    | 512 MB SDRAM             |
| <b>GPU</b>             | 250 MHz Videocore IV         | 250MHz Video core IV          | 400 MHz Video core IV         | 250MHz Video core IV     |
| <b>Storage</b>         | micro-SD                     | Micro-SD                      | micro-SD                      | micro-SD                 |
| <b>Ethernet</b>        | Yes                          | Yes                           | Yes                           | No                       |
| <b>Wireless</b>        | WiFi and Bluetooth           | No                            | No                            | No                       |

### **Raspberry Pi 3 Technical Specifications**

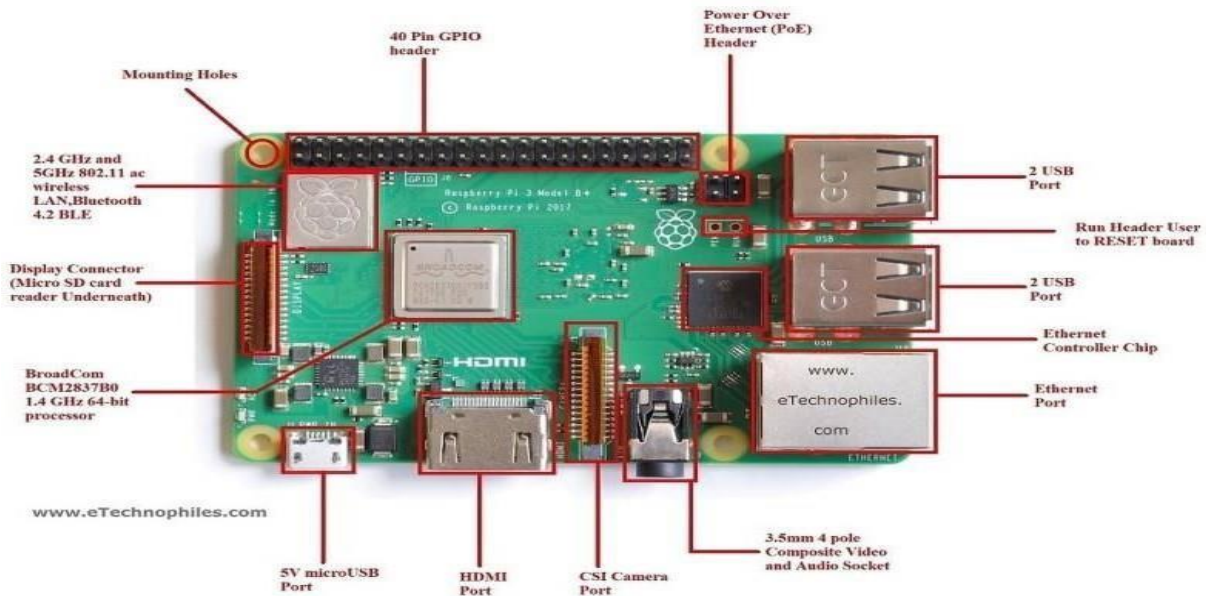
|                                                      |                                                                                                                                                                                                                                                    |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Microprocessor</b>                                | Broadcom BCM2837 64bit Quad Core Processor                                                                                                                                                                                                         |
| <b>Processor Operating Voltage</b>                   | 3.3V                                                                                                                                                                                                                                               |
| <b>Raw Voltage input</b>                             | 5V, 2A power source                                                                                                                                                                                                                                |
| <b>Maximum current through each I/O pin</b>          | 16mA                                                                                                                                                                                                                                               |
| <b>Maximum total current drawn from all I/O pins</b> | 54mA                                                                                                                                                                                                                                               |
| <b>Flash Memory (Operating System)</b>               | 16Gbytes SSD memory card                                                                                                                                                                                                                           |
| <b>Internal RAM</b>                                  | 1Gbytes DDR2                                                                                                                                                                                                                                       |
| <b>Clock Frequency</b>                               | 1.2GHz                                                                                                                                                                                                                                             |
| <b>GPU</b>                                           | Dual Core Video Core IV® Multimedia Co-Processor. Provides Open GLES 2.0, hardware-accelerated Open VG, and 1080p30 H.264 high-profile decode.<br><br>Capable of 1Gpixel/s, 1.5Gtexel/s or 24GFLOPs with texture filtering and DMA infrastructure. |
| <b>Ethernet</b>                                      | 10/100 Ethernet                                                                                                                                                                                                                                    |
| <b>Wireless Connectivity</b>                         | BCM43143 (802.11 b/g/n Wireless LAN and Bluetooth 4.1)                                                                                                                                                                                             |
| <b>Operating Temperature</b>                         | -40°C to +85°C                                                                                                                                                                                                                                     |

## Board Connectors

| Name                     | Description                                                                                           |
|--------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Ethernet</b>          | Base T Ethernet Socket                                                                                |
| <b>USB</b>               | 2.0 (Four sockets)                                                                                    |
| <b>Audio Output</b>      | 3.5mm Jack and HDMI                                                                                   |
| <b>Video output</b>      | HDMI                                                                                                  |
| <b>Camera Connector</b>  | 15-pin MIPI Camera Serial Interface (CSI-2)                                                           |
| <b>Display Connector</b> | Display Serial Interface (DSI) 15 way flat flex cable connector with two data lanes and a clock lane. |
| <b>Memory Card Slot</b>  | Push/Pull Micro SDIO                                                                                  |



## Raspberry Pi 3 Model B Board Description



**Processor & RAM:** Raspberry Pi is based on an ARM processor. **64** bit ARMv7Broadcom BCM2837 Quad Core Computer running at 1.2GHz ,

**USB Ports:** Raspberry Pi comes with FOUR 2.0 ports. The USB ports on Raspberry Pi can provide a current upto 100mA.

**Ethernet Ports:** Raspberry Pi comes with a standard RJ45 Ethernet port. We can connect an Ethernet cable or a USB WiFi adapter to provide Internet connectivity.

**HDMI Output:** The HDMI port on Raspberry Pi provides both video and audio output. We can connect the Raspberry Pi to a monitor using an HDMI cable. For monitors that have a DVI(Digital Video interface) port but no HDMI port, we can use an HDMI to DVI adapter/cable.

**Composite Video Output:** Raspberry Pi comes with a composite video output with an RCA jack that supports both PAL and NTSC video output. The RCA jack can be used to connect old televisions that have an RCA input only.

**Audio Output:** Raspberry Pi has a 3.5 mm audio output jack. This audio jack is used for providing audio output to old televisions along with the RCA jack for video. The audio quality from this jack is inferior to the HDMI output.

**GPIO Pins:** Raspberry Pi comes with a number of general purpose input/output pins.

Fig 3.3 shows the Raspberry Pi GPIO headers. There are four types of pins on Raspberry Pi- trueGPIO pins, I2C interface pins, SPI interface pins and serial Rx and Tx pins.

**Display Serial Interface(DSI):** The DSI interface can be used to connect an LCD panel to Raspberry Pi.

**Camera Serial Interface (CSI):** This interface can be used to connect a camera module to Raspberry Pi.

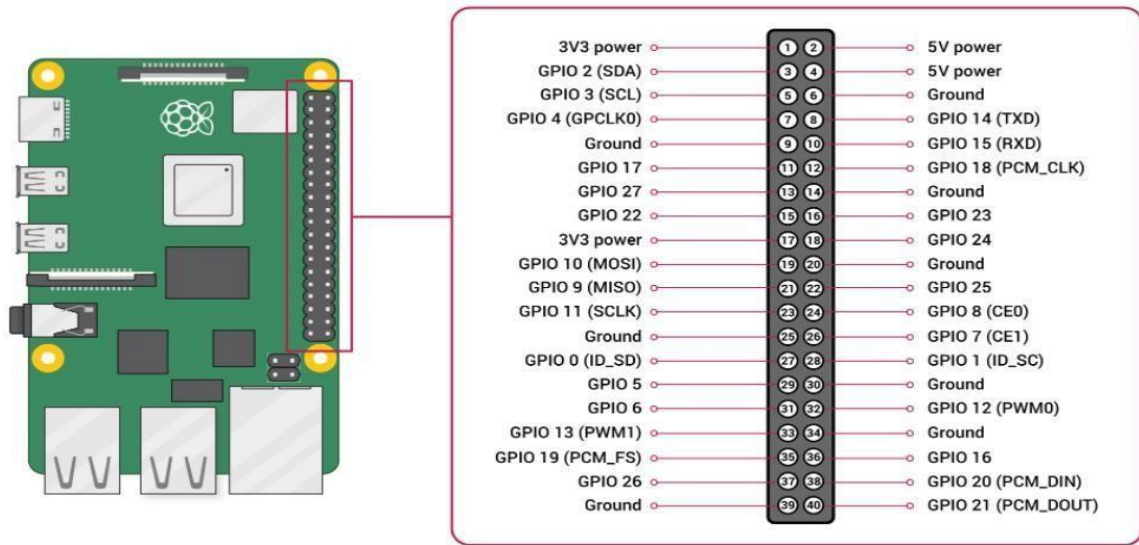
**Status LEDs:** Raspberry Pi has five status LEDs.

| Status LED | Function                  |
|------------|---------------------------|
| ACT        | SD card access            |
| PWR        | 3.3V Power is present     |
| FDX        | Full duplex LAN connected |
| LNK        | Link/Network activity     |
| 100        | 100 Mbit LAN connected    |

**SD Card Slot:** Raspberry Pi does not have a built in operating system and storage.

SD card slot on the Raspberry Pi 3 is located just below the Display Serial Adapter on the other side. We can plug-in an SD card loaded with a Linux image to the SD card slot.

**Power Input:** Raspberry Pi has a micro-USB connector for power input.



We require a external monitor you will require an HDMI cable to connect the monitor and the Raspberry pi you will require a keyboard and mouse a basic 5 volt adapter to power up the pi LAN cable and your memory card which will include the operating system on it.

### Raspberry Pi on board Serial Communication standards

#### UART:

UART is multi master communication protocol. This protocol is quite easy to use and very convenient for communicating between several boards: Raspberry Pi to Raspberry Pi, or Raspberry Pi to Arduino, etc.



For using UART you need 3 pins:

- GND that will connect to the global GND of your circuit.
- RX for Reception. It will connect this pin to the - TX pin of the other component.
- TX for Transmission. It will connect this pin to the RX of the other component.

If the component you're communicating with is not already powered, you'll also have to use a power pin (3.3V or 5V) to power on that component.

By using a UART to USB converter, you can communicate between your laptop and Raspberry Pi with UART.

### I2C:

I2C is a 2 WIRE master-slave bus protocol (it can have multiple masters but mostly used with one master and multiple slaves). The most common use of I2C is to read data from sensors and actuate some components.

The master is the Raspberry Pi, and the slaves are all connected to the same bus. Each slave has a unique ID, so the Raspberry Pi knows which component it should talk to.



### For using I2C you'll need 3 pins:

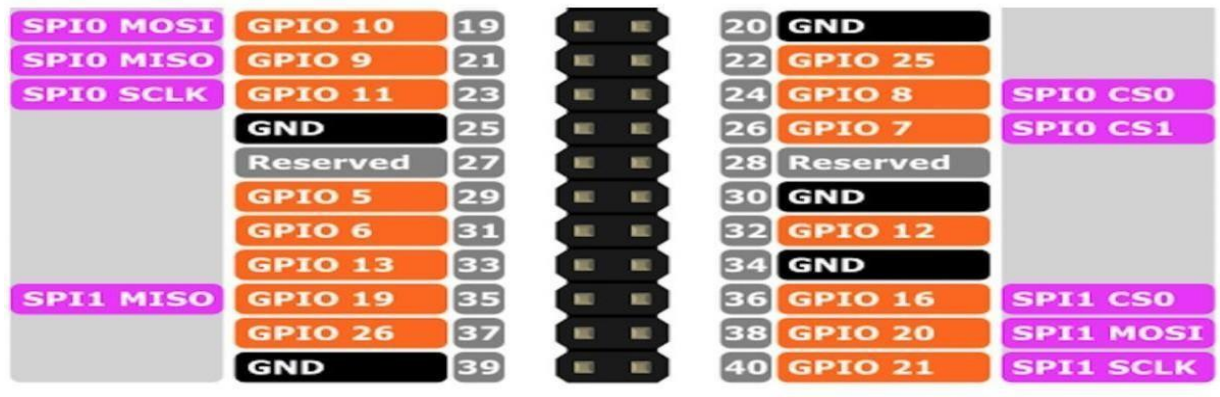
1. GND:
2. SCL: clock of the I2C. Connect all the slaves SCL to the SCL bus.
3. SDA: exchanged data. Connect all the slaves SDA to the SDA bus.
- 4.

Note that the SDA and SCL pins on the Raspberry Pi are alternate functions for GPIO 2 and 3. When you use a library (Python, Cpp, etc) for I2C, those two GPIOs will be configured so they can use their alternate function.

## SPI

Serial Peripheral Interface (SPI) is a synchronous serial data protocol used for communicating with one or more peripheral devices.

SPI is yet another hardware communication protocol. It is a master-slave bus protocol. It requires more wires than I2C, but can be configured to run faster.



As you can see, you get 2 SPIs by default: SPI0 and SPI1. It means you can use the Raspberry Pi as a SPI master on two different SPI buses at the same time.

And, as for I2C, SPI uses the alternate functions of GPIOs.

For using SPI you'll need 5 pins:

1. GND: what a surprise! Make sure you connect all GND from all your slave components and the Raspberry Pi together.
2. SCLK: clock of the SPI. Connect all SCLK pins together.
3. MOSI: means Master Out Slave In. This is the pin to send data from the master to a slave.
4. MISO: means Master In Slave Out. This is the pin to receive data from a slave to the master.
5. CS: means Chip Select. Pay attention here: you'll need one CS per slave on your circuit. By default you have two CS pins (CS0 – GPIO 8 and CS1 – GPIO 7). You can configure more CS pins from the other available GPIOs.

## Raspberry Pi Python Basics

### 1 .Python RPi.GPIO API Library

- One of the famous library in launching input and output pins is the RPi.GPIO library. You can use this library in python programming.
- Use the [RPi.GPIO module](#) as the driving force behind our Python GPIO examples. This set of Python files and source is included with Raspbian OS --NOOBS,(Linux based )

### 2. How to Setup RPi.GPIO

- This library is available on Raspbian operating system by default and you don't need to install it.
- In order To use RPi.GPIO in Python script, you need to put this statement at the top of your file: **import RPi.GPIO as GPIO**
- That statement "includes" the RPi.GPIO module, and goes a step further by providing a local name -- GPIO -- which we'll call to reference the module from here on.

### 3. Pin Numbering Declaration

- After including the RPi.GPIO module, the next step is to determine which of the two **pin-numbering schemes**
- 1. **GPIO.BOARD** : Board numbering scheme. The pin numbers follow the pin numbers on header P1.
- 2. **GPIO.BCM** : Broadcom chip-specific pin numbers. These pin numbers follow the lower-level numbering system defined by the Raspberry Pi's Broadcom-chip brain.
- To specify in your code which number-system is being used, use the **GPIO.setmode()function**.

**Example:**

**GPIO.setmode(GPIO.BCM)***#will activate the Broadcom-chip specific pin numbers.*

**Note: Both the import and setmode lines of code are required, if you want to use Python.**

**4. Setting a Pin Mode:**

- If you've used Arduino, you're probably familiar with declare a "pin mode" before you can use it as either an input or output.
- In Raspberry Pi to set a pin mode, use the

**GPIO.setup(pin number, pinmode) function.**

- **Example:** if you want to set pin 18 as an output, write:
- **GPIO.setup(18, GPIO.OUT)**

**Note: Remember that the pin number will change if you're using the board numbering system (instead of 18, it'd be 12).**

**GPIO.setmode(GPIO.BCM);** Using pins BCM numbers

**Basic Commands**

- **GPIO.setup(pin, GPIO.IN);** Determines the pin as input
- **GPIO.setup( pin, GPIO.OUT);** Determines the pin as an output
- **GPIO.input(pin):** Reading input pin
- **GPIO.output( pin, state):** Writing on the output pin.

## **Experiment 1: Controlling LED ON/OFF with Arduino/ESP32/Raspberry Pi**

**AIM:** To interface & write a python program to Control LED ON/OFF with Arduino/ESP32/ Raspberry Pi.

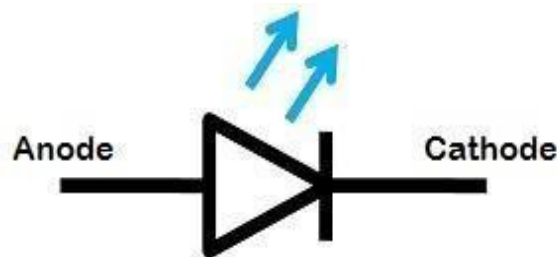
### **Components Required:**

- Raspberry Pi/Arduino UNO
- LED
- Resistors 470Ω
- Jumper wires

### **Theory:**

#### **LED:**

- The LED is a special type of diode and they have similar electrical characteristics to a PN junction diode. Hence the LED allows the flow of current in the forward direction and blocks the current in the reverse direction.
- The LED symbol is similar to a diode symbol except for two small arrows that specify the emission of light, thus it is called LED (light-emitting diode). The LED includes two terminals namely anode (+) and the cathode (-). The LED symbol is shown below.



### **Working of LED**

- When the voltage is not applied to the LED, then there is no flow of electrons and holes so they are stable. Once the voltage is applied then the LED will forward biased, so the electrons in the N-region and holes from P- region will move to the active region. This region is also known as the depletion region. Because the charge carriers like holes include a positive charge whereas electrons have a negative charge so the light can be generated through the recombination of polarity charges



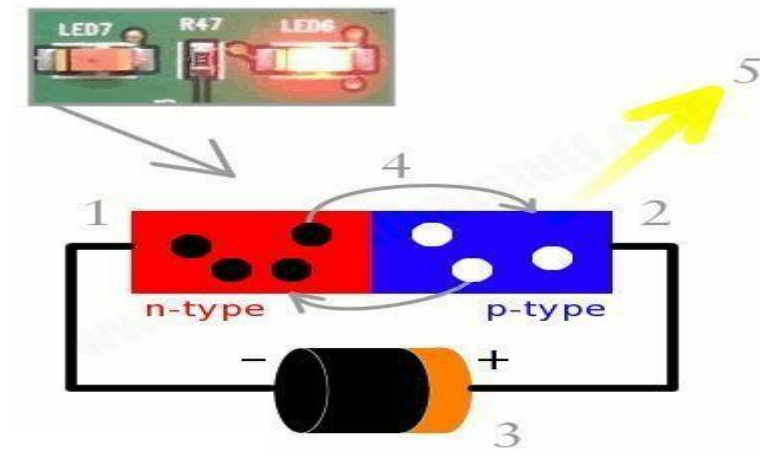


Figure: Working of Light Emitting Diode

## 2. Resistor:

- A resistor is a passive two-terminal electrical component that implements electrical resistance as a circuit element.
- In electronic circuits, resistors are used to reduce current flow, adjust signal levels, to divide voltages, bias active elements, and terminate transmission lines, among other uses.

### Symbol of Resistor

- Resistor is a 2 terminal passive device. The symbol is given below.

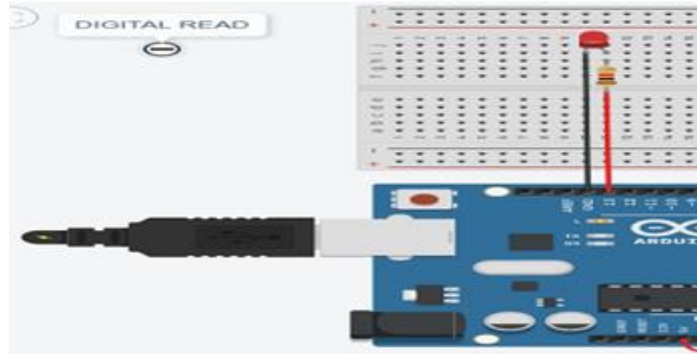


Figure: Resistor Symbol

### Unit of resistance

- The SI-unit of resistance is Ohm [ $\Omega$ ]. The higher multiple and sub-multiple values of ohm are kilo ohms [ $K\Omega$ ], mega ohms [ $M\Omega$ ], milli ohm and so on

## Interfacing LED with Arduino



### Program:

```
void setup() {  
  pinMode(13, OUTPUT); // sets the digital pin 13 as output  
}  
  
void loop() {  
  digitalWrite(13, HIGH); // sets the digital pin 13 on  
  delay(1000);           // waits for a second  
  digitalWrite(13, LOW); // sets the digital pin 13 off  
  delay(1000);           // waits for a second  
}
```

### OUTPUT:

### RESULT:

**Viva:**

1. What is an LED? Draw the symbol.

Ans:

2. What is the working principle of LED?

Ans:.

3. Give some applications of LED module?

Ans:.

4. What is Baud Rate?

Ans.

5. What is the importance of ICSP header?

Ans:

## **Experiment 2: Implementation of Automated Street Lighting System** **using LDR with Arduino/ESP32/Raspberry Pi**

**AIM:** To interface & write a program to implement Automated Street Lighting System using LDR with Arduino.

### **Components Required:**

- Arduino UNO
- LDR
- Resistors 470 $\Omega$ , 10k $\Omega$
- Jumper wires

### **LDR (Light Dependent Resistor)**

A Light Dependent Resistor (LDR) or a photo resistor is a device whose resistivity is a function of the incident electromagnetic radiation. Hence, they are light sensitive devices. They are also called as photo conductors, photo conductive cells or simply photocells.



**Program :**

```
int ldr=A0;//Set A0(Analog Input) for LDR.

int value=0;

void setup() {

Serial.begin(9600);pinMode(3,OUTPUT);

}

void loop() {
value=analogRead(ldr);//Reads the Value of LDR(light).

Serial.println("LDR value is :");//Prints the value of LDR to SerialMonitor.

Serial.println(value);

if(value<300)

{

digitalWrite(3,HIGH);//Makes the LED glow in Dark.

}

else

{

digitalWrite(3,LOW);//Turns the LED OFF in Light.

}

}
```

**OUTPUT****RESULT:**

**Viva:**

1. What is an LDR?

Ans:

2. What is the difference between analog and digital signals? Mention with example?

Ans:

3. Mention Analog pins of Arduino?

Ans:

4. Arduino has how many bits in-built ADC?

Ans:

5. What is Serial.begin(9600)?

Ans:

### **Experiment 3 : Monitoring the voltage level of the battery and indicating the same using multiple LED's & OLED with Arduino/ESP32/Raspberry Pi.**

**AIM:** To interface & write a program to monitor battery voltage level and indicating on OLED display and multiple LED's.

#### **Equipment Required:**

- Arduino UNO
- OLED Display
- Resistors 1k $\Omega$ , 10K $\Omega$
- LED card
- 9V Hi-watt Battery
- Jumper wires

#### **Theory:**

##### **OLED:**

The organic light-emitting diode (OLED) display SSD1306 model: a monicolor, 0.96-inch display with 128 $\times$ 64 pixels as shown in the following figure.



The OLED display doesn't require backlight, which results in a very nice contrast in dark environments. Additionally, its pixels consume energy only when they are on, so the OLED display consumes less power when compared with other displays.

The model we're using here has only four pins and communicates with the Arduino using I2C

communication protocol. There are models that come with an extra RESET pin. There are also other OLED displays that communicate using SPI communication.

### Pin wiring

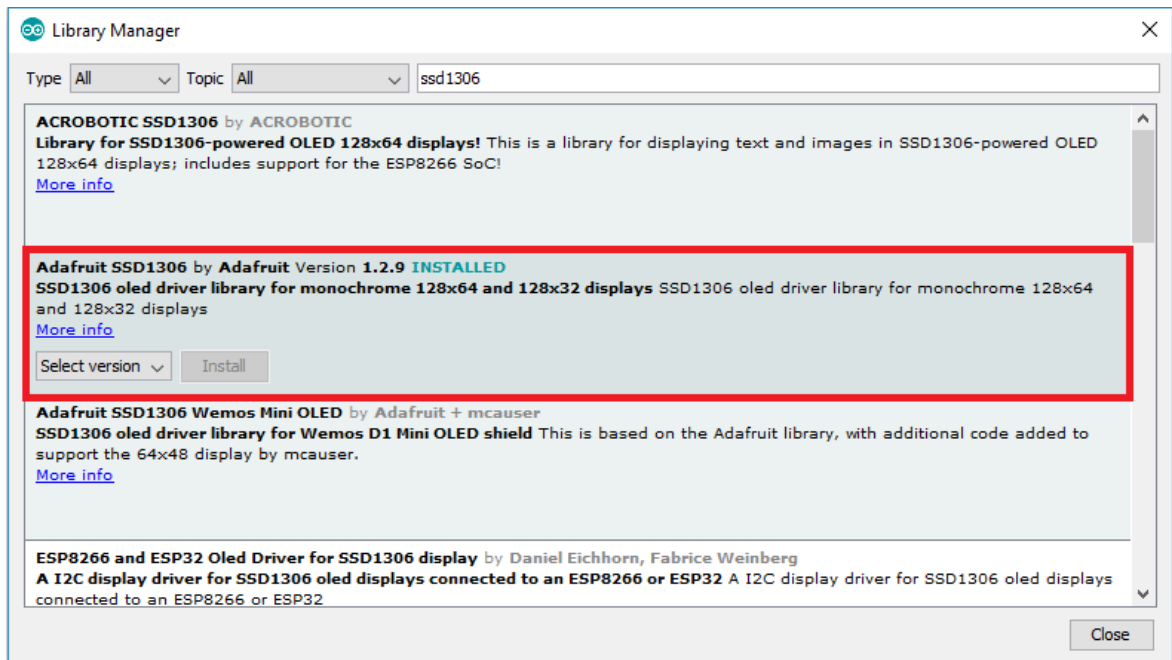
Because the OLED display uses I2C communication protocol, wiring is very simple. You just need to connect to the Arduino Uno I2C pins as shown in the table below.

| Pin | Wiring to Arduino Uno |
|-----|-----------------------|
| Vin | 5V                    |
| GND | GND                   |
| SCL | A5                    |
| SDA | A4                    |

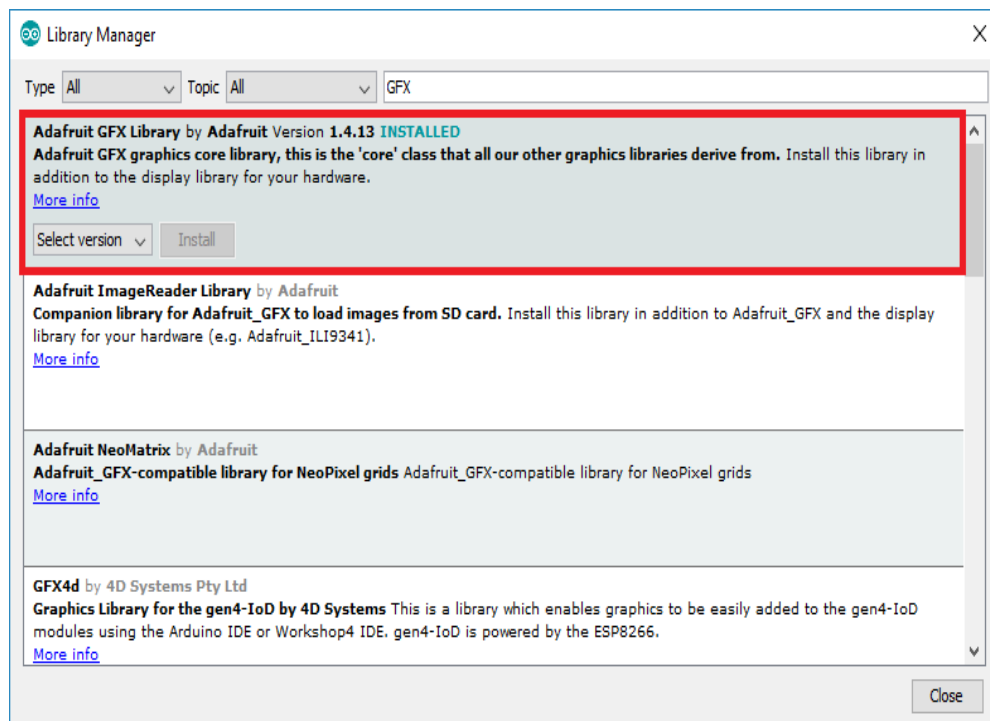
To control the OLED display you need the adafruit\_SSD1306.h and the adafruit\_GFX.h libraries. Follow the next instructions to install those libraries.

1. Open your Arduino IDE and go to **Sketch > Include Library > Manage Libraries**. The Library Manager should open.
2. Type “**SSD1306**” in the search box and install the SSD1306 library from Adafruit.





3. After installing the SSD1306 library from Adafruit, type “GFX” in the search box and install the library.



4. After installing the libraries, restart your Arduino IDE.

**Program:**

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#define OLED_RESET 4
Adafruit_SSD1306 display(OLED_RESET);

int greenLed = 2;
int yellowLed = 3;
int redLed = 4;
int analogValue = 0;
float voltage = 0;
int ledDelay = 1000;

void setup()
{

display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
display.display();
display.clearDisplay();
Serial.begin(9600);
pinMode(greenLed, OUTPUT);
pinMode(yellowLed, OUTPUT);
pinMode(redLed, OUTPUT);
}

void loop()
{
analogValue = analogRead(A0);

voltage = 0.0048*analogValue;
display.setTextSize(1);
display.setTextColor(WHITE);
display.setCursor(0,0);
display.println(" Battery Voltage");
display.println("");
display.println("");
display.setTextSize(3);
display.print(" ");
display.println(voltage);
display.display();
delay (200);
```

```
display.clear Display();  
Serial.println(voltage);  
if( voltage >= 1.35 )  
    digitalWrite(greenLed, HIGH);  
else if (voltage > 1.2 && voltage < 1.35)  
    digitalWrite(yellowLed, HIGH);  
else if( voltage <= 1.2)  
    digitalWrite(redLed, HIGH);  
delay(ledDelay);  
digitalWrite(redLed, LOW);  
digitalWrite(yellowLed, LOW);  
digitalWrite(greenLed, LOW);  
}
```

**OUTPUT:**

**RESULT:**

**Viva:**

1. What is an OLED?

Ans:

2. What is OLED SSD1306 model?

Ans:

3. What is I2C?

Ans:

4. What is Adafruit SSD1306?

Ans:.

5. Instruction `display.setTextColor(WHITE)` indicates.?

Ans:

## Experiment 4: Generation of a random value similar to dice game and display the same using a 16X2 LCD with Arduino/ESP32/Raspberry Pi

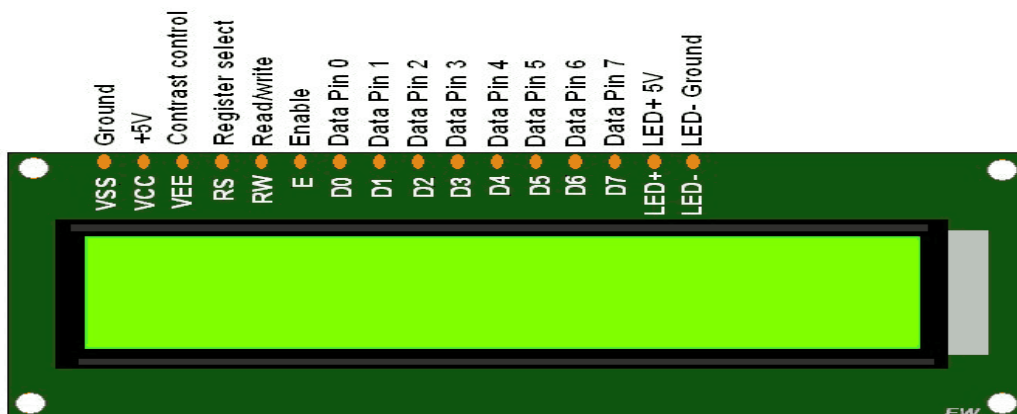
**AIM:** To interface & write a program to generate a random value similar to dice game simulation and display on LCD using Arduino.

### **Components Required:**

- Arduino UNO
- 16X2 LCD
- Push button switch
- Resistor 470 $\Omega$ , 1k $\Omega$ , 10K $\Omega$
- Jumper wires

### **LCD(Liquid Crystal Display)**

- LCDs (Liquid Crystal Displays) are used in embedded system applications for displaying various parameters and status of the system.
- LCD 16x2 is a 16-pin device that has 2 rows that can accommodate 16 characters each. LCD 16x2 can be used in 4-bit mode or 8-bit mode.
- It is also possible to create custom characters.
- It has 8 data lines and 3 control lines that can be used for control purposes.

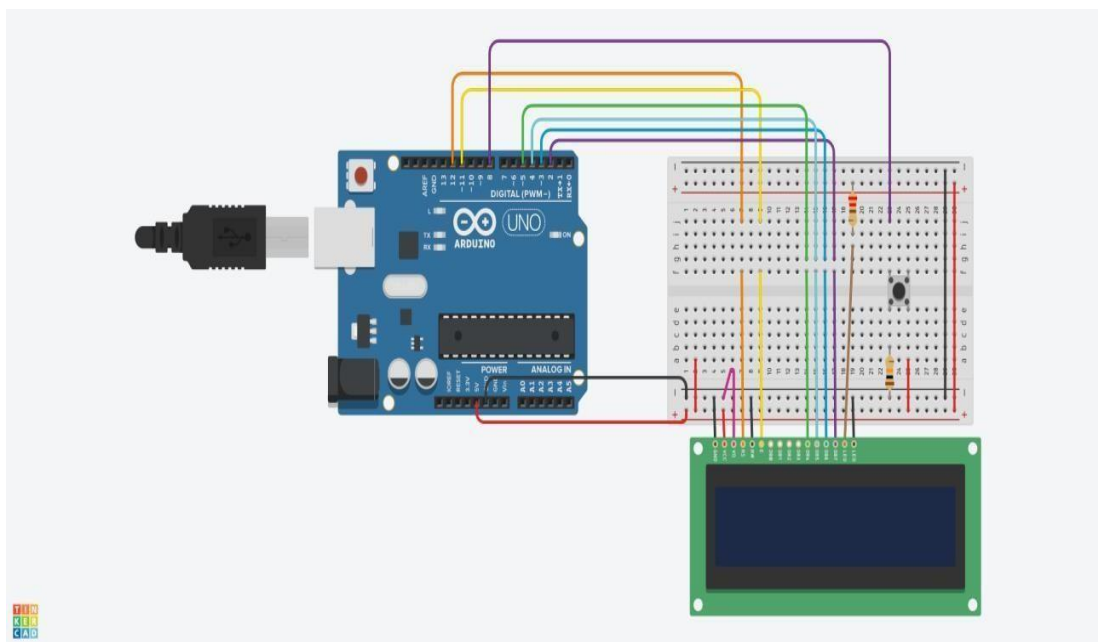


| Pin No | Function                                                       | Name            |
|--------|----------------------------------------------------------------|-----------------|
| 1      | Ground (0V)                                                    | Ground          |
| 2      | Supply voltage; 5V (4.7V – 5.3V)                               | Vcc             |
| 3      | Contrast adjustment; through a variable resistor               | VEE             |
| 4      | Selects command register when low; and data register when high | Register Select |
| 5      | Low to write to the register; High to read from the register   | Read/write      |
| 6      | Sends data to data pins when a high to low pulse is given      | Enable          |
| 7      | 8-bit data pins                                                | DB0             |
| 8      |                                                                | DB1             |
| 9      |                                                                | DB2             |
| 10     |                                                                | DB3             |
| 11     |                                                                | DB4             |
| 12     |                                                                | DB5             |
| 13     |                                                                | DB6             |
| 14     |                                                                | DB7             |
| 15     | Backlight VCC (5V)                                             | Led+            |
| 16     | Backlight Ground (0V)                                          | Led-            |

## LCD Command Codes

| Code (Hex) | Command to LCD Instruction Register      |
|------------|------------------------------------------|
| 1          | Clear display screen                     |
| 2          | Return home                              |
| 4          | Decrement cursor (shift cursor to left)  |
| 6          | Increment cursor (shift cursor to right) |
| 5          | Shift display right                      |
| 7          | Shift display left                       |
| 8          | Display off, cursor off                  |
| A          | Display off, cursor on                   |
| C          | Display on, cursor off                   |
| E          | Display on, cursor blinking              |
| F          | Display on, cursor blinking              |
| 10         | Shift cursor position to left            |
| 14         | Shift cursor position to right           |
| 18         | Shift the entire display to the left     |
| 1C         | Shift the entire display to the right    |
| 80         | Force cursor to beginning to 1st line    |
| C0         | Force cursor to beginning to 2nd line    |
| 38         | 2 lines and 5x7 matrix                   |

Schematic:



**Program:**

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
int randomnumber;

void setup()
{
    // put your setup code here, to run once:
    lcd.begin(16, 2);
    randomSeed(7);
    pinMode(8,
    INPUT);
}

void loop()
{
    lcd.setCursor(2, 0);
    lcd.print("Dice value
    is:");
    int DICEROLL =

    digitalRead(8);

    if(DICEROLL==1)

    randomnumber = random(1,7);
    lcd.setCursor(6, 1);
    lcd.print(randomnumber);
}
```

**OUTPUT:****RESULT:**



**Viva:**

1. What is an LCD?

Ans:

2. What is an RS in LCD pin Configuration?

Ans:

3. 16X2 LCD indicates?

Ans:

4. Instruction `lcd.setCursor(6, 1)` indicates?

Ans:

5. What is the length of Data Bus in 16x2 LCD?

Ans:

## **Experiment 5: Print temperature and humidity using DHT22 sensor with Raspberry Pi /Arduino/ESP32**

**AIM:** To interface & write a program to print temperature and humidity using DHT22sensor with Arduino.

### **Components Required:**

- Arduino UNO
- DHT22
- Resistors 1k $\Omega$
- Jumper wires

### **Theory:**

#### **DHT22:**

**Temperature:** A measure of the warmth or coldness of an object or substance with reference to some standard value.

**Humidity:** A quantity representing the amount of water vapour in the atmosphere.

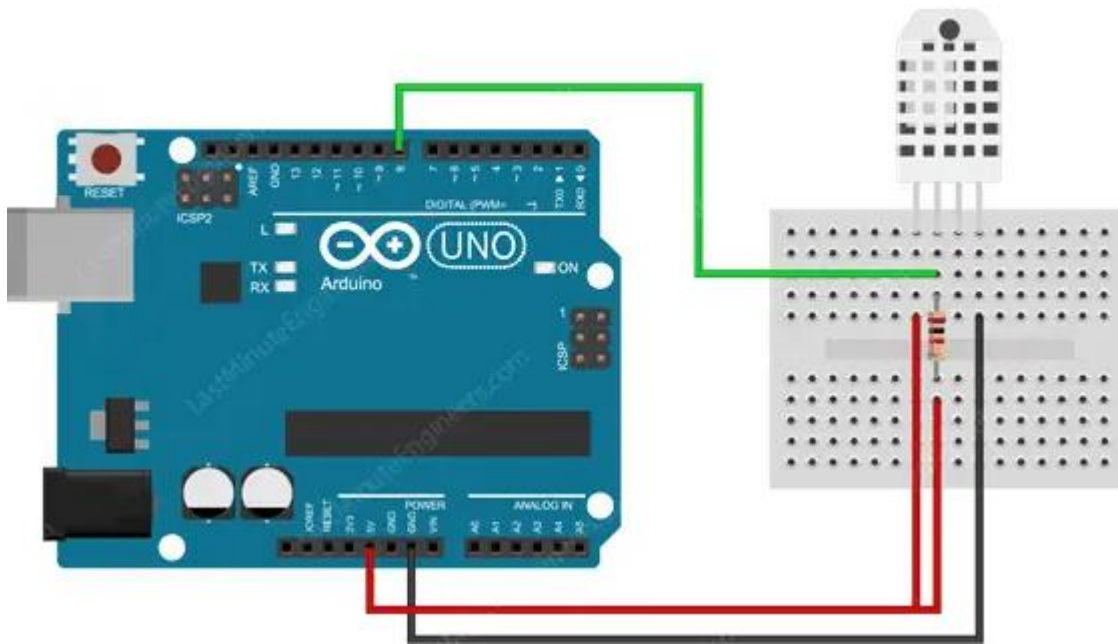
### **DHT22: Digital Humidity Temperature sensor**

- The DHT-22 (also named as AM2302) is a digital-output relative humidity and temperature sensor. It uses a capacitive humidity sensor and a thermistor to measure the surrounding air, and spits out a digital signal on the data pin.

### **Features:**

- The range of operating voltage ( ) is 3 V to 5 V power.
- It measures temperature in a range of -40°C to +125°C with an accuracy of  $\pm 0.5$  degrees.
- The measuring range for humidity is from 0 to 100% with an accuracy of 2-5%.
- The maximum operating current for DHT22 sensor is 2.5mA.
- The sampling rate for DHT22 sensor is 0.5 Hz. It takes measurement once every 2seconds.

## Pin Description:



The DHT22 sensor is very simple and easy to use. It has only four pins.

1. **Vcc** is the power pin. Apply voltage in a range of 3.5 V to 5.0 V at this pin.
2. **Data Out** is the digital output pin. It sends out the value of measured temperature and humidity in the form of serial data.
3. **N/C** is a not connect pin.
4. **GND**: Connect the GND pin to main ground.



**Figure: Pin Configuration of DHT22**

**Program:**

```
#include <DHT.h>

// Define the DHT sensor type and pin
#define DHTPIN 8    // Digital pin connected to the DHT sensor
#define DHTTYPE DHT22 // DHT 22 (AM2302)

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  Serial.println("DHT22 Temperature and Humidity Sensor");
  dht.begin();
}

void loop() {
  // Delay between readings (adjust as needed)
  delay(2000);

  // Read temperature and humidity
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();

  // Print the readings to the serial monitor
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.print(" *C");
  Serial.print(" Humidity: ");
  Serial.print(humidity);
  Serial.println(" %");
}
```

**OUTPUT:**

**RESULT:**

**Viva:**

1. What is DHT22?

Ans:

2. Name the library which is required to work with DHT22?

Ans:

3. What is Data Out?

Ans:

4. What is the range of operating Voltage of DHT22?

Ans:

5. What is the range of operating temperature & accuracy of DHT22?

Ans:

## **Experiment 6: Controlling of PIR based LED ON/OFF**

**AIM:** To interface & write a python program to Control PIR based LED ON/OFF with RaspberryPi

### **Components required:**

- RaspberryPi3b+ board
- 1 HDMI Probe
- Keyboard
- Mouse
- Monitor
- SD Card with NOOBSOS
- PIR
- LED

### **PIR SENSOR**

- A passive infrared sensor (PIR sensor) is an electronic sensor that measures infrared (IR) light radiating from objects in its field of view.(range)
- PIR sensors are commonly used in security alarms and automatic lighting applications.
- PIR sensors detect general movement, but do not give information on who or what moved.

### **PIN DESCRIPTION**

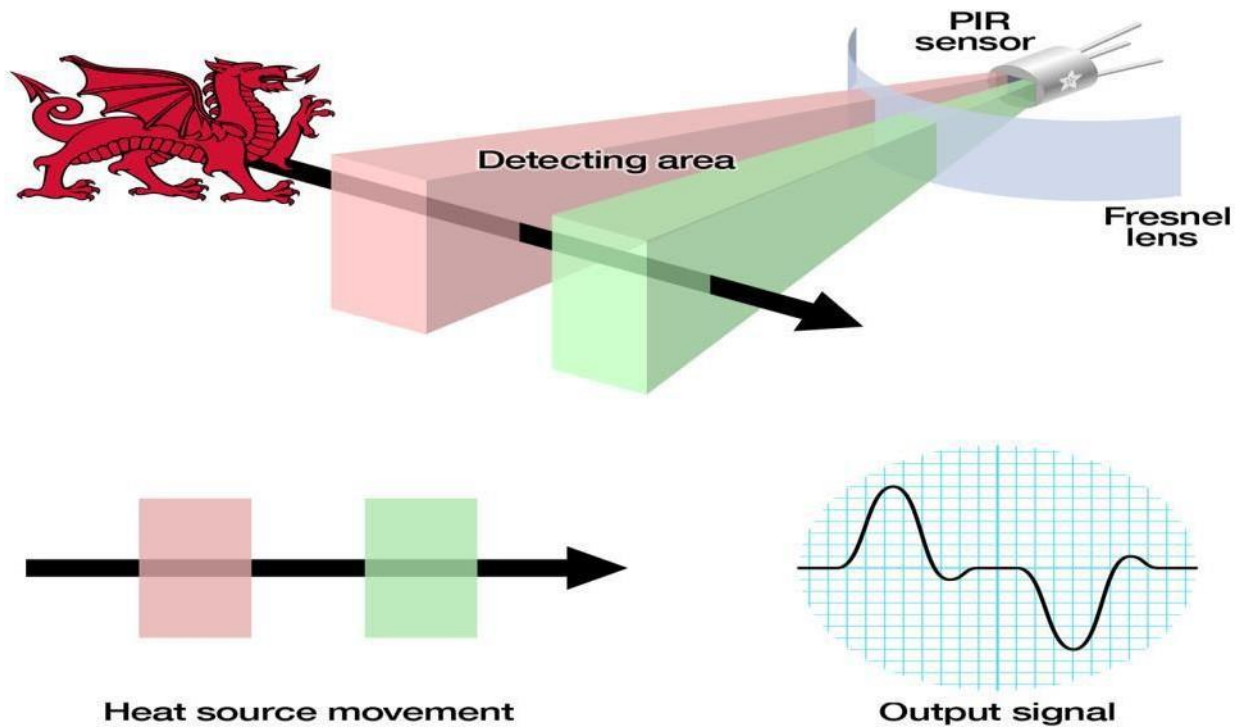
- **Pin1** corresponds to the drain terminal of the device, which connected to the positive supply 5V DC.
- **Pin2** corresponds to the source terminal of the device, which connects to the ground terminal via a 100K or 47K resistor. The Pin2 is the output pin of the sensor.
- **Pin3** of the sensor connected to the ground.

### **PIR WORKING**

- The PIR sensor itself has two slots in it, each slot is made of a special material that is sensitive to IR. When the sensor is idle, both slots detect the same amount of IR, the ambient amount radiated from the room or walls or outdoors.
- When a warm body like a human or animal passes by, it first intercepts one half



of the PIR sensor, which causes a positive differential change between the two halves. When the warm body leaves the sensing area, the reverse happens, whereby the sensor generates a negative differential change. These change pulses are what is detected.



**Figure: illustration of PIR working**

**Program:**

```
import RPi.GPIO as GPIO
import time
```

```
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BOARD)
GPIO.setup(11, GPIO.IN)
GPIO.setup(3, GPIO.OUT)
```

```
#Read ouput from PIR motion sensor
#LED output pin
```

While True:

```
i=GPIO.input(11)
```

```
if i == 0:
```

```
#When output from motion sensor is LOW
```

```
    print ("No Person detected",i)  
    GPIO.output(3,0)  
    time.sleep(0.1)
```

```
elif i == 1:
```

```
#When output from motion sensor is HIGH
```

```
    print ("A Person is detected",i)  
    GPIO.output(3,1)  
    time.sleep(0.1)
```

```
#Turn ON LED
```

**OUTPUT:**

**RESULT:**

**Viva:**

1. What is PIR sensor?

Ans:

2. Explain the working of PIR sensor?

Ans:

3. What is Applications of PIR sensor?

Ans:

4. Can a PIR sensor detects whether object is Mouse or human being?

Ans:

5. Explain the Pin functionality of PIR?

Ans:

## **Experiment 7: Implementation of Traffic Light system with Raspberry Pi**

**AIM:** To interface & write a python program to implement Traffic Lighting System with Raspberry Pi.

### **Components required:**

- RaspberryPi3b+ board
- 1 HDMI Probe
- Keyboard
- Mouse
- Monitor
- SD Card with NOOBSOS
- Traffic Light LED Module

### **Traffic Light System:**



Traffic lights are an essential part of today's road infrastructure. Traffic lights are a set of three lights that signal whether one grouping of Vehicles can go or if they need to stop by combining these lights in series at an intersection a smooth flow of traffic can be ensured. Green signals "GO", Yellow or Amber signals to "Slow Down and Prepared to Stop" and Red means "Stop". As a vehicle approaches a traffic light at a typical intersection it has the choice of turning left, right or going straight. These choices have to be addressed by the signal from the traffic light.

**Program:**

```
import RPi.GPIO as GPIO
import time
GPIO.setmode(GPIO.BCM)

GPIO.setup(2,GPIO.OUT)
GPIO.setup(3,GPIO.OUT)
GPIO.setup(4,GPIO.OUT)

GPIO.setup(17,GPIO.OUT)
GPIO.setup(27,GPIO.OUT)
GPIO.setup(22,GPIO.OUT)

GPIO.setup(10,GPIO.OUT)
GPIO.setup(9,GPIO.OUT)
GPIO.setup(11,GPIO.OUT)

GPIO.setup(13,GPIO.OUT)
GPIO.setup(6,GPIO.OUT)
GPIO.setup(5,GPIO.OUT)
while True:

    GPIO.output(2,False)
    GPIO.output(3,False)
    GPIO.output(4,False)

    GPIO.output(17,False)
    GPIO.output(27,False)
    GPIO.output(22,False)

    GPIO.output(10,False)
    GPIO.output(9,False)
    GPIO.output(11,False)

    GPIO.output(5,False)
    GPIO.output(6,False)
    GPIO.output(13,False)
    time.sleep(2)
```

```
print(" Green Signal for Lane 2 & Lane 4 & Red signal for Lane 1 & Lane 3")
GPIO.output(2,True)
GPIO.output(10,True)
GPIO.output(13,True)
GPIO.output(22,True)
time.sleep(8)
print(" Yellow Signal for Lane 2 & Lane 4" )

GPIO.output(2,False)
GPIO.output(10,False)
GPIO.output(13,False)
GPIO.output(22,False)
GPIO.output(27,True)
GPIO.output(6,True)
time.sleep(6)

print(" Green Signal for Lane 1 & Lane 3 & Red signal for Lane 2 & Lane 4")
GPIO.output(27,False)
GPIO.output(6,False)
GPIO.output(17,True)
GPIO.output(5,True)
GPIO.output(4,True)
GPIO.output(11,True)
time.sleep(8)

print(" Yellow Signal for Lane 1 & Lane 3" )
GPIO.output(17,False)
GPIO.output(5,False)
GPIO.output(4,False)
GPIO.output(11,False)
GPIO.output(9,True)
GPIO.output(3,True)
time.sleep(4)
```

**OUTPUT:**

**RESULT:**



**Viva:**

1. What are the applications of Traffic Light System?

Ans:

2. What is the importance of time module?

Ans:

3. What are the advantages of Traffic Light System?

Ans:

## **Experiment 8: Measure the distance using Ultra sonic sensor**

**AIM:** To interface & write a python program to measure the distance using ultra sonic sensor with Raspberry Pi.

### **Components required:**

- RaspberryPi3b+board/ESP32
- 1 HDMI Probe
- Keyboard
- Mouse
- Monitor
- SDCard with NOOBSOS
- Ultra sonic sensor
- Dustbin

### **Theory:**

An ultrasonic sensor is an electronic device that measures the distance of a target object by emitting ultrasonic sound waves, and converts the reflected sound into an electrical signal.



The HC-SR04 is a type of ultrasonic sensor which uses sonar to find out the distance of the object from the sensor. It provides an outstanding range of non-contact detection with high accuracy & stable readings. It includes two modules like ultrasonic transmitter & receiver. This sensor is used in a variety of applications like measurement of direction and speed, burglar alarms, medical, sonar, humidifiers, wireless charging, non-destructive testing, and ultrasonography.

## HC-SR04 Ultrasonic Sensor Pin Configuration

This sensor includes four pins and the pin configuration of this sensor is discussed below.

- Pin1 (Vcc): This pin provides a +5V power supply to the sensor.
- Pin2 (Trigger): This is an input pin, used to initialize measurement by transmitting ultrasonic waves by keeping this pin high for 10 $\mu$ s.
- Pin3 (Echo): This is an output pin, which goes high for a specific time period and it will be equivalent to the duration of the time for the wave to return back to the sensor.
- Pin4 (Ground): This is a GND pin used to connect to the GND of the system.

### Features:

- The power supply used for this sensor is +5V DC
- Dimension is 45mm x 20mm x 15mm
- Quiescent current used for this sensor is <2mA
- The input pulse width of trigger is 10 $\mu$ s
- Operating current is 15mA
- Measuring angle is 30 degrees
- The distance range is 2cm to 800 cm
- Resolution is 0.3 cm
- Effectual Angle is <15°
- Operating frequency range is 40Hz
- Accuracy is 3mm
- The HC-SR04 Ultrasonic sensor comes with four pins namely Vcc pin, Trigger pin, Echo pin, & Ground pin. This sensor is used to measure the accurate distance between the target and the sensor. This sensor mostly works on the sound waves.
- When the power supply is given to this module, it generates the sound waves to travel throughout the air to hit the necessary object. These waves strike and come back from the object, then collect by the receiver module.
- Here both the distance as well as time has taken is directly proportional because the time taken for more distance is high. If the trigger pin is kept high for 10  $\mu$ s, then the ultrasonic waves will be generated which will travel at the sound speed. So it creates eight cycles of sonic burst that will be gathered within the Echo pin. This ultrasonic sensor is interfaced with Arduino to gauge the necessary distance between sensor & object. The distance can be calculated using the following formula.

- $S = (V \times t)/2$

- Where the 'S' is the required distance
- 'V' is the sound's speed
- 't' is the time taken for sound waves to return back after striking the object.
- The actual distance can be calculated by dividing its value with 2 as the time will be twice once the waves travel and get back from the sensor.

### **Applications:**

**The applications of HC-SR04 sensor** include the following,

- This sensor is used to measure speed as well as the direction between two objects
- It is used in wireless charging
- Medical ultrasonography
- This is used to detect objects & avoid obstacles using robots such as biped, path finding, obstacle avoidance, etc.
- Depth measurement
- Humidifiers
- This sensor is used to plot the objects nearby the sensor by revolving it
- Non-destructive testing
- By using this sensor depth of pits, wells can be measured by transmitting the waves through water.
- Embedded system
- Burglar alarms

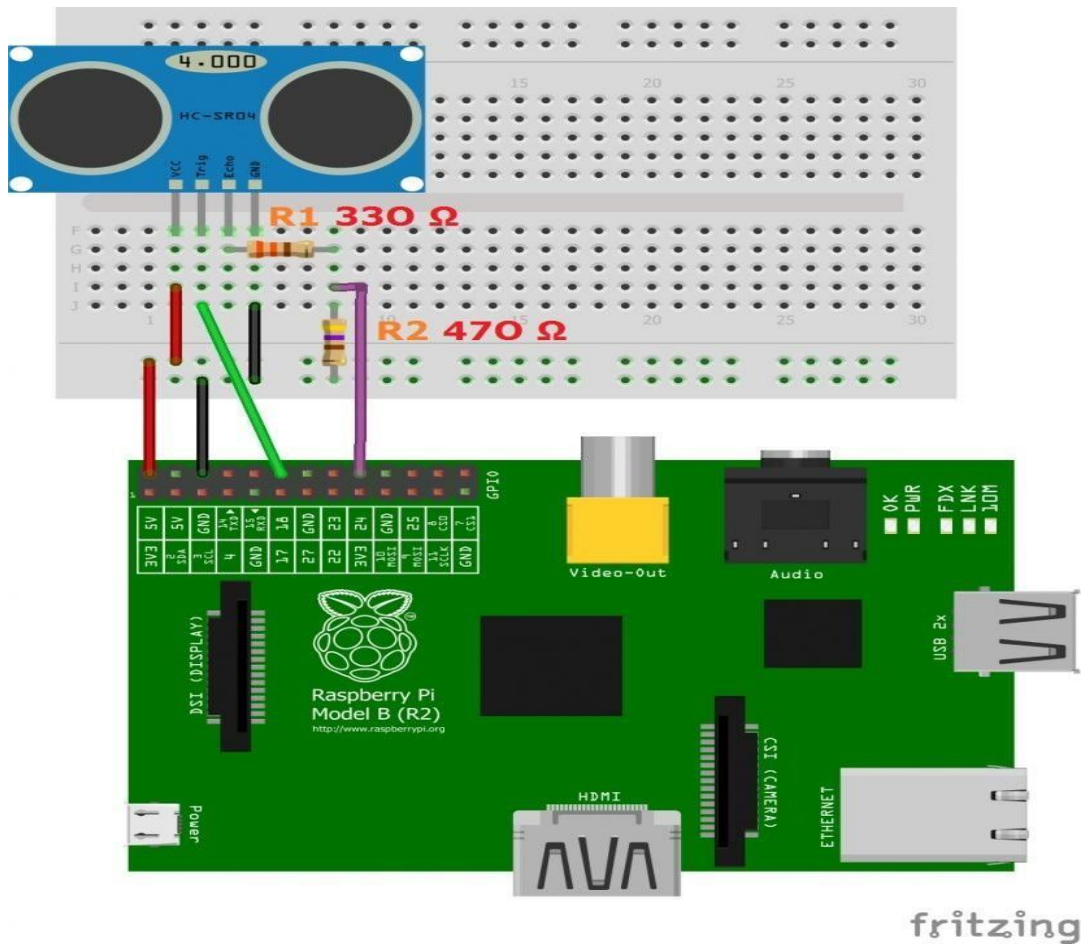
### **Using a Raspberry Pi distance sensor (Ultrasonic sensor HC-SR04)**

For many (outdoor) projects a distance measurement is necessary or advantageous. These can measure the distance up to 4-5 meters.

#### **Wiring**

There are four pins on the ultrasound module that are connected to the Raspberry:

- VCC to Pin 2(VCC)
- GND to Pin 6(GND)
- TRIG to Pin12(GPIO18)
- Connect the 330Ω resistor to ECHO. On it send you connect it to Pin18 (GPIO24) and through a 470Ω resistor you connect it also to Pin6(GND).



### Program:

```
#Libraries
import RPi.GPIO as GPIO
import time

#GPIOMode(BOARD/ BCM)
GPIO.setmode(GPIO.BCM)

#set GPIO Pins
GPIO_TRIGGER=18
GPIO_ECHO=24

#set GPIO direction (IN / OUT)
GPIO.setup(GPIO_TRIGGER,GPIO.OUT)
GPIO.setup(GPIO_ECHO,GPIO.IN)

def distance():
```

```

    # set Trigger to HIGH
    GPIO.output(GPIO_TRIGGER,True)

    # set Trigger after 0.01ms to LOW
    time.sleep(0.00001)
    GPIO.output(GPIO_TRIGGER,False)

    StartTime = time.time()
    StopTime=time.time()

    # save StartTime
    while GPIO.input(GPIO_ECHO) == 0:
        StartTime=time.time()

    # save time of arrival
    while GPIO.input(GPIO_ECHO) == 1:
        StopTime=time.time()

    # time difference between start and arrival
    TimeElapsed=StopTime-StartTime
    #multiply with the sonicspeed(34300cm/s)
    #and divide by2, because there and back
    distance = (TimeElapsed * 34300) / 2

    return distance

if __name__ == '__main__':
    try:
        while True:
            dist=distance()
            print ("Measured Distance = %.1f cm" % dist)
            time.sleep(1)
        # Reset by pressingCTRL + C
    except KeyboardInterrupt:
        print("Measurement stopped by User")
    GPIO.cleanup()

```

**OUTPUT:**

**Result:**

**Viva:**

1. What is an Ultra Sonic Sensor?

Ans:

2. Maximum Distance measured by HR-SR04?

Ans:

3. Explain the Functionality of PIN2 & PIN3 of HR-SR04?

Ans:

4. Mention the Application of ultrasonic sensor?

Ans:



## **Experiment 9: Perform image capture, read, write, and modify the image operation** **using Pi camera with Raspberry Pi**

**AIM:** To perform image capture, read, write, and modify the image operations using Pi camera with Raspberry Pi

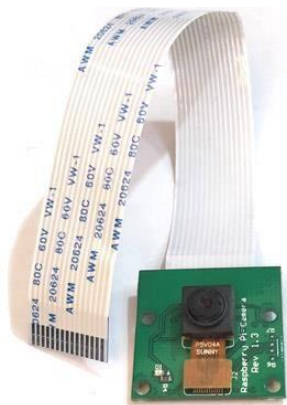
### **Components required:**

- RaspberryPi3b+board
- 1 HDMI Probe
- Keyboard
- Mouse
- Monitor
- SD Card with NOOBSOS
- PI Camera

### **PI camera:**

The Pi camera module is **a portable light weight camera that supports Raspberry Pi**. It communicates with Pi using the MIPI camera serial interface protocol. It is normally used in image processing, machine learning or in surveillance projects.

Pi Camera module can be used **to take pictures and high definition video**. Raspberry Pi Board has CSI (Camera Serial Interface) interface to which we can attach Pi Camera module directly using 15-pin ribbon cable.



**Program:**

```
#capture the image

from picamera import PiCamera
from time import sleep

camera = PiCamera()
camera.rotation = 180
camera.start_preview()
sleep(5)
camera.stop_preview()

#modify the image

from picamera import PiCamera
from time import sleep
from picamera import PiCamera, Color

camera = PiCamera()
camera.rotation = 180
camera.start_preview()
camera.image_effect = 'watercolor'
camera.annotate_background = Color('yellow')
camera.annotate_foreground = Color('blue')
camera.annotate_text_size = 80
camera.annotate_text = "Hello KMIT"

for i in range(5):
    sleep(5)
    camera.capture('/home/pi/Desktop/Rasp1%s.jpg' % i)
    camera.stop_preview()

#video

from picamera import PiCamera
from time import sleep
from picamera import PiCamera, Color

camera = PiCamera()
camera.rotation = 180
camera.start_preview()
```

```
camera.start_preview()
camera.annotate_background = Color('yellow')
camera.annotate_foreground = Color('blue')
camera.annotate_text_size = 80

camera.annotate_text = " KMIT IT "
camera.start_recording('/home/pi/Desktop/video.h264')
sleep(5)
camera.stop_recording()
camera.stop_preview()
```

**OUTPUT:**

**RESULT:**

### **Viva**

1. What is Pi camera module?

Ans:

2. What is the instruction to show the image using Raspberry Pi?

Ans:

3. Which instruction is used to capture the image using Raspberry Pi?

Ans:

4. How to display message on screen?

Ans:

## **Experiment-10: Creating a firebase cloud account and connect to cloudservices to monitor device data.**

**AIM:** To create a firebase cloud account and write a program to connect cloud services to monitor device data with Raspberry Pi

### **Components required:**

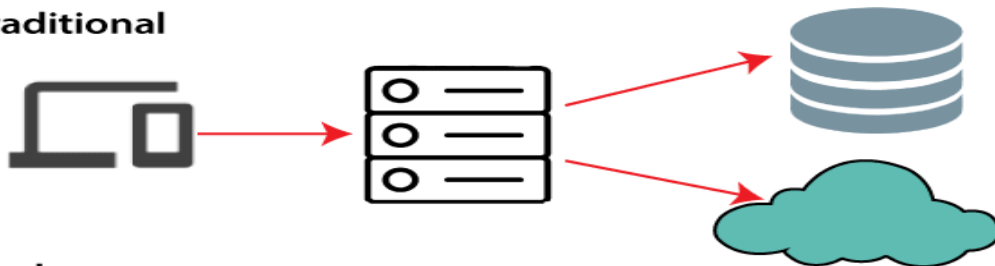
- RaspberryPi3b+board
- 1 HDMI Probe
- Keyboard
- Mouse
- Monitor
- SD Card with NOOBSOS

### **Firebase:**

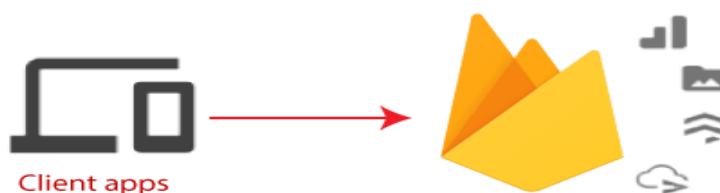
In the era of rapid prototyping, we can get bright ideas, but sometimes they are not applicable if they take too much work. Often, the back-end is the limiting factor - many considerations never apply to server-side coding due to lack of knowledge or time.

Firebase is a Backend-as-a-Service(BaaS) which started as a YC11 startup. It grew up into a next-generation app-development platform on Google Cloud Platform. Firebase (a NoSQL JSON database) is a real-time database that allows storing a list of objects in the form of a tree. We can synchronize data between different devices.

#### **Traditional**

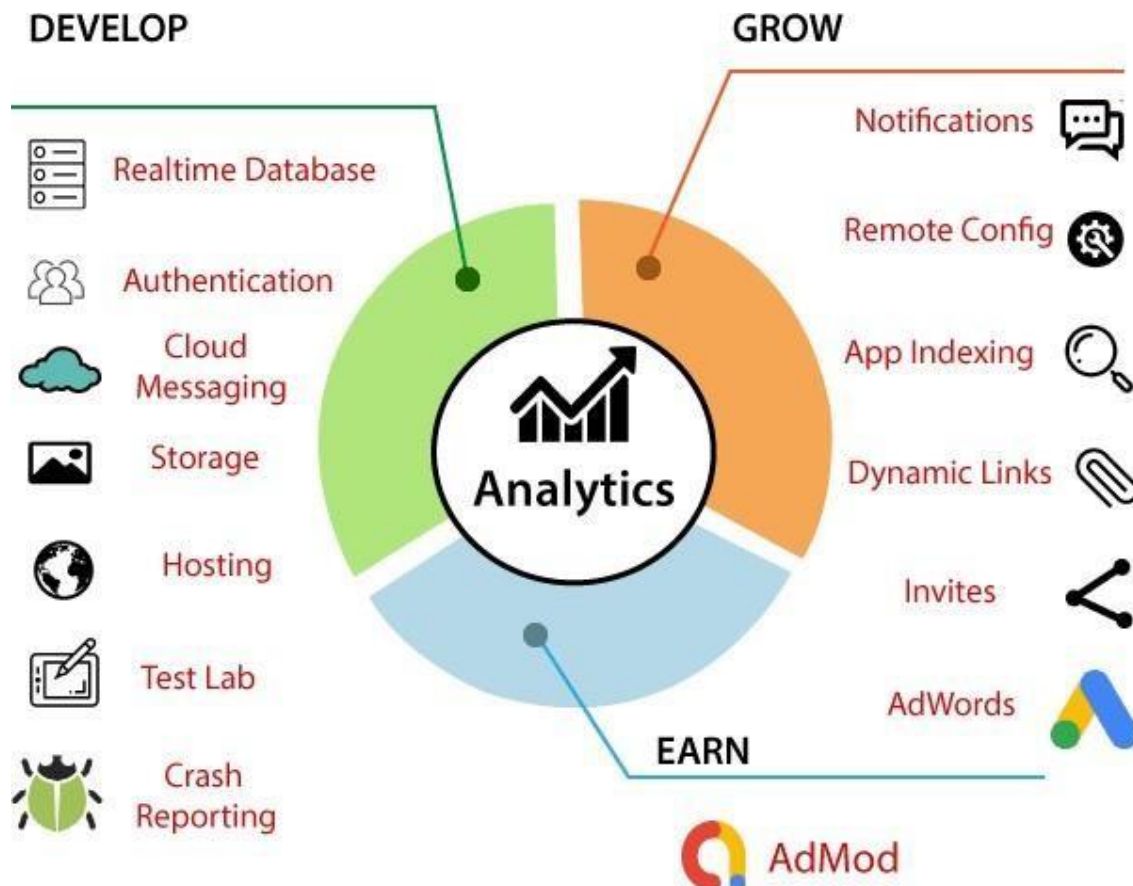


#### **Firebase**



Google Firebase is Google-backed application development software which allows developers to develop **Android, IOS, and Web apps**. For reporting and fixing app crashes, tracking analytics, creating marketing and product experiments, firebase provides several tools.

Firebase has three main services, i.e., a real-time database, user authentication,



and hosting. We can use these services with the help of the Firebase iOS SDK to create apps without writing any server code.

**Why use Firebase:**

- Firebase manages real-time data in the database. So, it easily and quickly exchanges the data to and from the database. Hence, for developing mobile apps such as live streaming, chat messaging, etc., we can use Firebase.
- Firebase allows syncing real-time data across all devices - iOS, Android, and Web –without refreshing the screen.
- Firebase provides integration to Google Advertising, AdMob, Data Studio, BigQueryDoubleClick, Play Store, and Slack to develop our apps with efficient and accurate management and maintenance.
- Everything from databases, analytics to crash reports are included in Firebase. So, the app development team can stay focused on improving the user experience.
- Firebase applications can be deployed over a secured connection to the firebase server.
- Firebase offers a simple control dashboard.
- It offers a number of useful services to choose from.

**Pros and Cons of Firebase:**

Firebase has a lot of pros or advantages. Apart from the advantages, it has disadvantages too. Let's take a look at the dvantagesand disadvantages:

**Pros:**

- Firebase is a real-time database.
- It has massive storage size potential.
- Firebase is serverless.
- It is highly secure.
- It is the most advanced hosted BaaS solution.
- It has minimal setup.
- It provides three-way data binding via angular fire.
- It provides simple serialization of app state.
- We can easily access data, files, auth and more.

- There is no server infrastructure required to power apps with data.
- It has JSON storage, which means no barrier between data and objects.

### **Cons:**

- Firebase is not widely used, or battle-tested for enterprises.
- It has very limited querying and indexing.
- It provides no aggregation.
- It has no map-reduce functionality.
- It cannot query or list users or stored files.

### **Procedure to Create Firebase Account:**

#### **Step1**

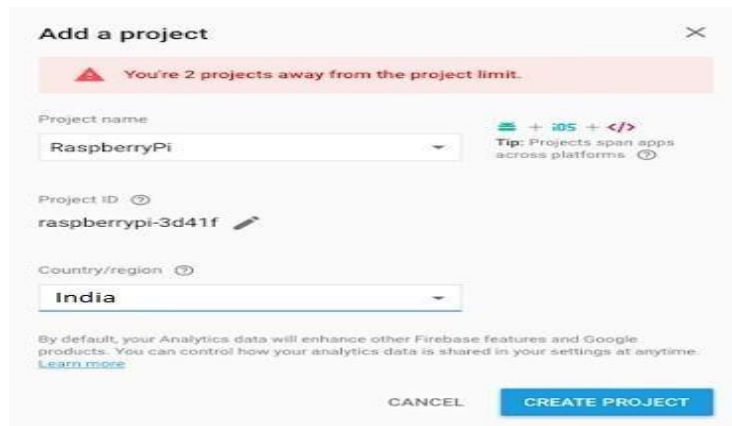
Login to your Firebase account (<https://firebase.google.com/>), then go to the Firebase console application.





## Step2:

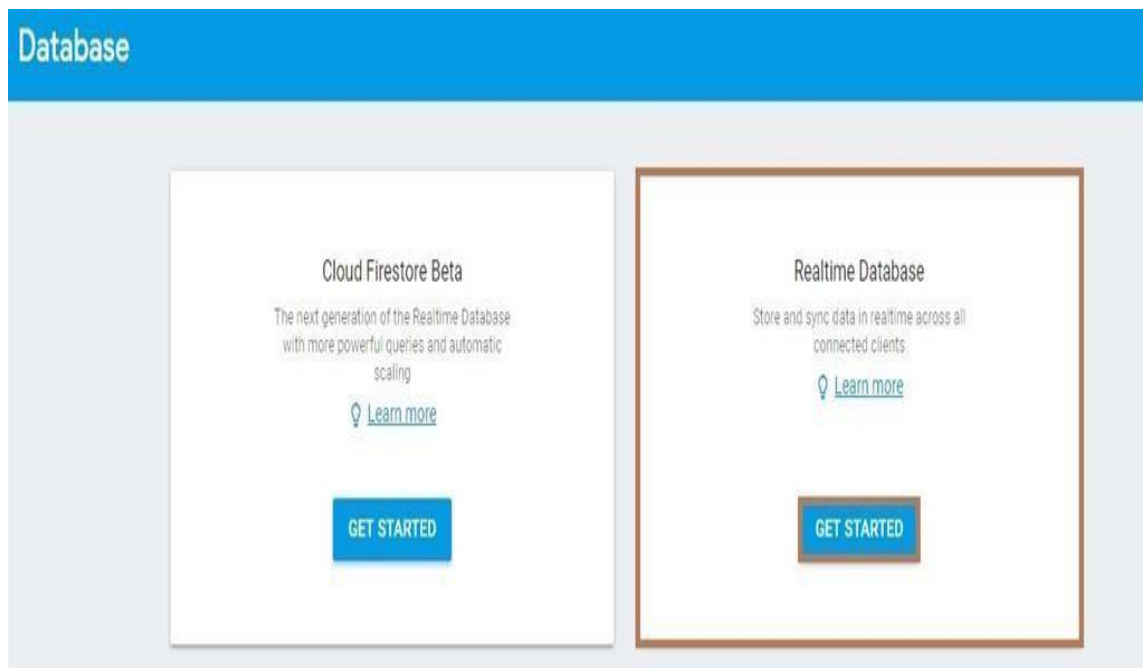
Create a new project and fill in the necessary details.



The screenshot shows the 'Add a project' dialog box. At the top, there is a warning message: 'You're 2 projects away from the project limit.' Below this, the 'Project name' field is set to 'RaspberryPi'. To the right of this field is a tip: 'Tip: Projects span apps across platforms.' Below the project name, the 'Project ID' is 'raspberrypi-3d41f'. The 'Country/region' is set to 'India'. At the bottom, there are two buttons: 'CANCEL' and 'CREATE PROJECT'.

## Step3

- Next, goto select Develop and then click on Database in your project.
- After that, the Database appears and select the "Real-time Database GetStarted" button.



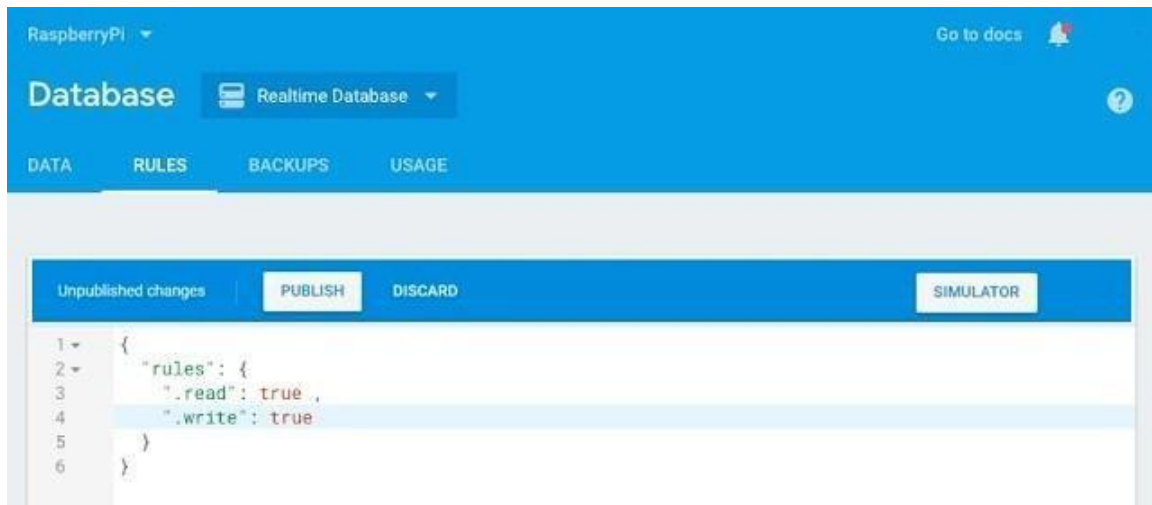
## Step4

Next, `gotorules>>changerules`. Just replace that with the following code.

### Default Rules

```
1.// require authentication
2.{
3.  "rules":{
4.    ".read":"auth!=null",
5.    ".write":"auth!=null"
6.  }
7. }
```

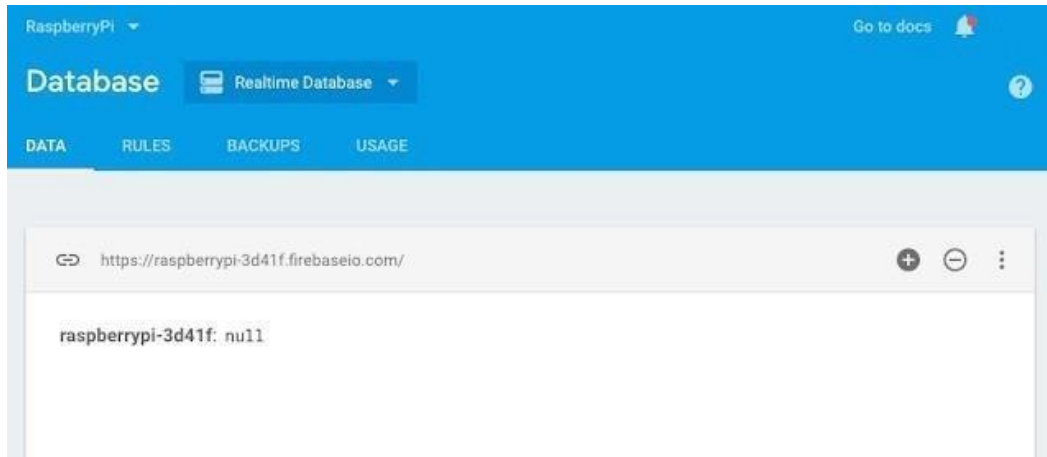
### Change to these rules



This is for the first test. Anyone can read and write your database without authentication.

```
1.//Not require authentication
2.{
3.  "rules":{
4.    ".read":true,
5.    ".write":true
6.  }
7. }
```

- Apply the above code to Firebase rule, then click the publish button to deploy the rules. After the rules are published anyone can read and write to your data.
- Goto database>>real-timedatabase>> data



### Step5

Open the python IDE, File >> New. Save the name as dht-firebase.py.py, then follow the givencode.

#### Program:

```
import RPi.GPIO as GPIO
GPIO.setwarnings(False)

GPIO.setmode(GPIO.BOARD)
#GPIO.setup(10,GPIO.IN,pull_up_down=GPIO.PUD_DOWN)
from datetime import datetime
#from picamera import PiCamera
from time import sleep

import osimport
pyrebasefirebaseConfig={
    'apiKey':"AIzaSyABtUYpOCaczxhfx1MNEtPX70ohAKDk1EY",'authDomain':"rpi-
testing-e3379.firebaseio.com",
    'databaseURL':"https://rpi-testing-e3379-default-rtdb.firebaseio.com",'projectId':"rpi-
testing-e3379",
    'storageBucket':"rpi-testing-e3379.appspot.com", 'messagingSenderId':"943281949971",
    'appId':
    "1:943281949971:
web:2b765d4d08eba7c5dadee6",'measurementId':"G-TMFQNXZCHE"
}
```

```

firebase = pyrebase.initialize_app(firebaseConfig)

storage= firebase.storage()
#db=firebase.database()
#db is to send the sensor data of irebase.
#db.child("<NAME>").set(<sensor data>)

#To constantly update the sensor data in the cloud storage
#db.child("<NAME>").push(<sensor data>)

#To push the data to the storage name= "google.jpeg"
    #camera.capture(name)
    #print(name+"saved")
storage.child(name).
put(name)
print("Image sent")
os.remove(name)
print("FileRemoved")
sleep(2)

```

**Output:**

**Result:**

**Viva:**

1. What is Firebase

Ans:

2. Mention the few Advantages of Firebase?

Ans:

•

3. Mention the few Disadvantages of Firebase?

Ans:

4. Creating Google Firebase is Free?

Ans:

⋮

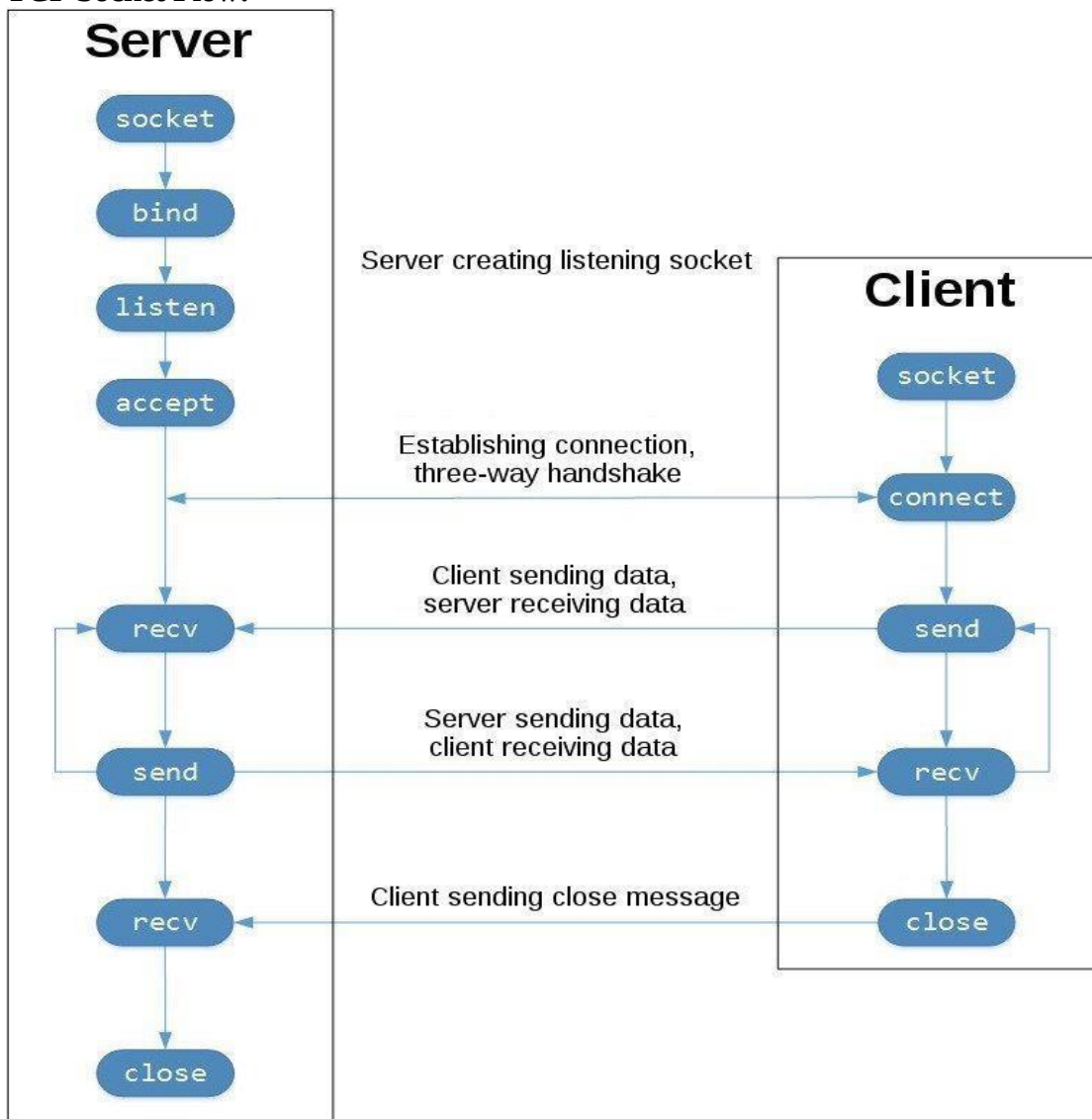
## Experiment 11: Monitor data remotely using TCP/UDP Protocol

**AIM:** To monitor data remotely using TCP/UDP Protocol with Raspberry Pi

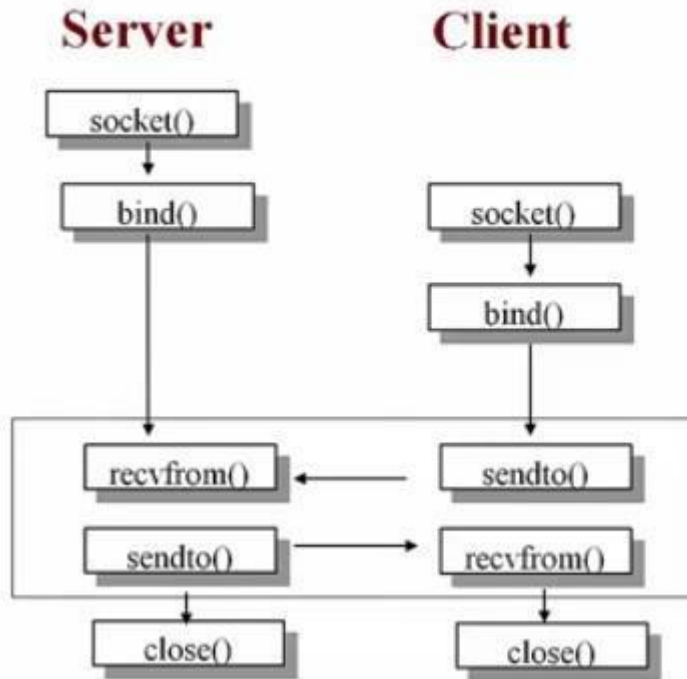
### **Sockets:**

Sockets are the endpoints of a bidirectional communications channel. Sockets may communicate within a process, between processes on the same machine, or between processes on different continents. Sockets may be implemented over a number of different channel types: Unixdomain sockets, TCP, UDP, and so on.

### **TCP Socket Flow:**



# Connectionless Protocol



## Socket Programming:

- Creates a two-way communication between two nodes in a network
- The nodes are termed as Server and Client

## Creating a socket:

**`s = socket.socket (SocketFamily, SocketType, Protocol=0)`**

- SocketFamily can be AF\_UNIX or AF\_INET
- SocketType can be SOCK\_STREAM or SOCK\_DGRAM
- Protocol is set default to 0

## Server Program

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Bind the socket to the port
server_address = ("192.168.0.2",2000)
sock.bind(server_address)

while True:
    data,address = sock.recvfrom(4096)
    with open("Datalog.txt","a") as f:
        mess=str(data)
        f.write(mess)
        print(mess)
    f.close()
```

## Client Program:

```
import socket
import sys
import RPi.GPIO as GPIO
from time import sleep
import Adafruit_DHT

def sensordata():
    GPIO.setmode(GPIO.BCM)
    GPIO.setwarnings(False)
    Sensor=Adafruit_DHT.DHT22
    humidity, temperature = Adafruit_DHT.read_retry(Sensor,4)
    return(humidity, temperature)

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) #create UDP socket
server_address = ("192.168.43.234",2000)
```



try:

```
while (1):
```

```
    h,t = sensordata()
```

```
    message = str(h)+'/'+str(t)
```

```
    #Send data
```

```
    print (sys.stderr, 'sending "%s"' % message)
```

```
    sent = sock.sendto(message.encode(), server_address)
```

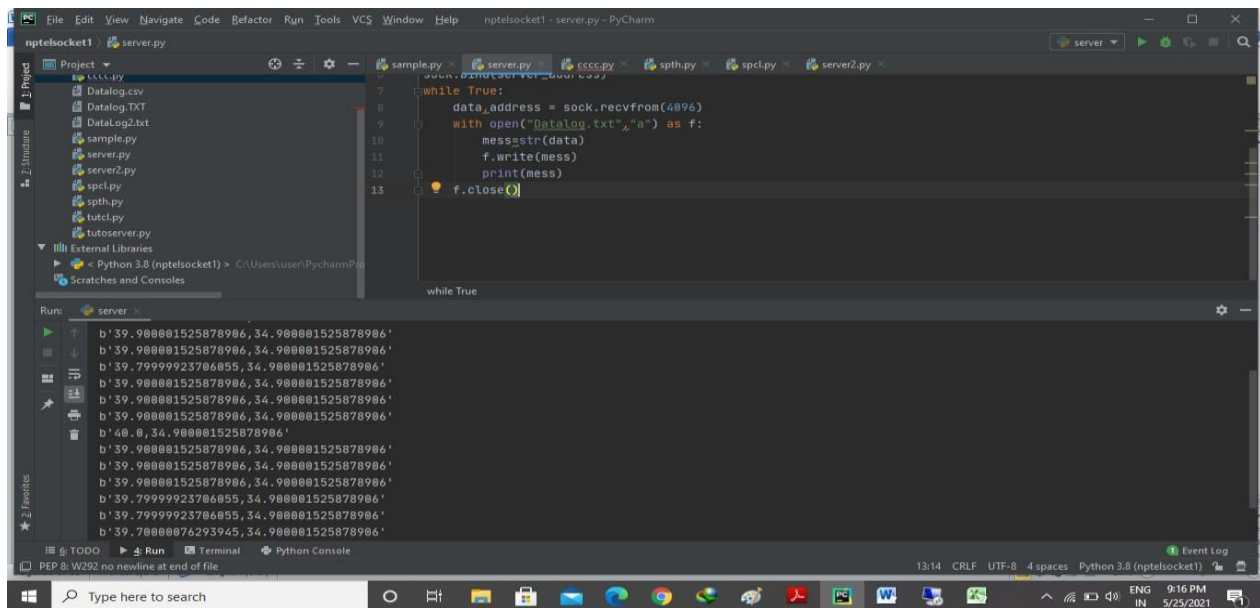
finally:

```
    print (sys.stderr, 'closing socket')
```

```
    sock.close()
```

## OUTPUT:

### SERVER Receiving Data:



The screenshot shows the PyCharm IDE interface. The top pane displays the code for `server.py`, which is a simple socket server that receives data from a client and writes it to a file named `Datalog.txt`. The code is as follows:

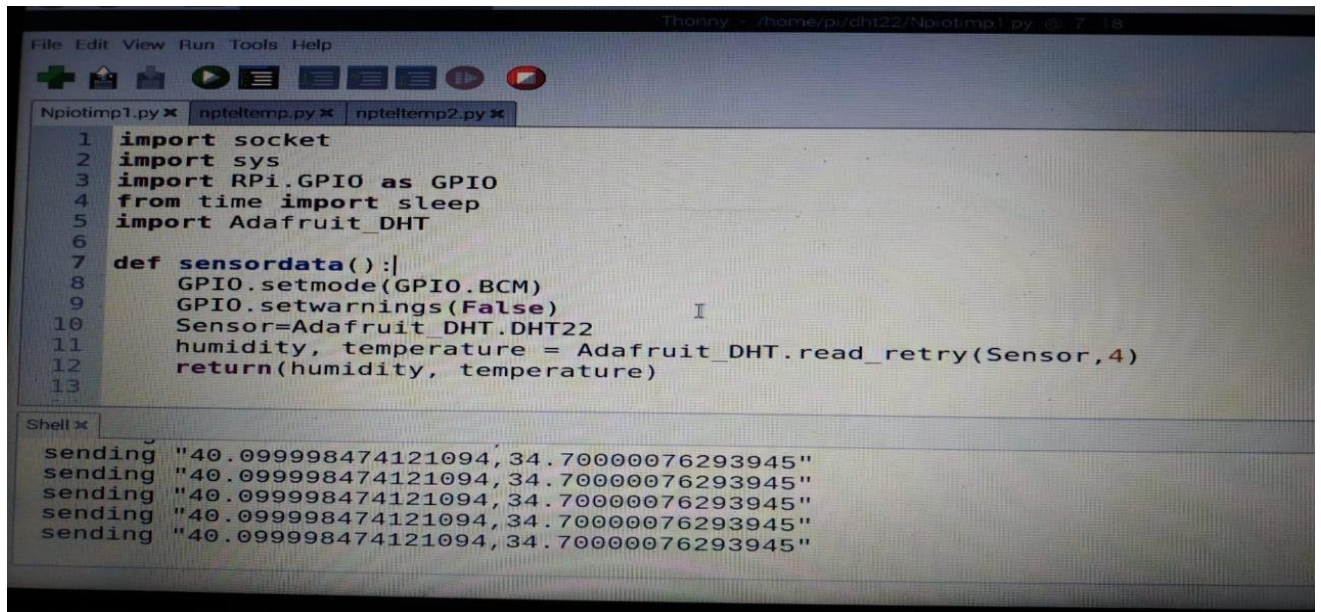
```
import socket
import sys
import os

server_address = ('localhost', 8080)
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind(server_address)
sock.listen(1)

while True:
    data, address = sock.recvfrom(4096)
    with open("Datalog.txt", "a") as f:
        mess = str(data)
        f.write(mess)
        print(mess)
    f.close()
```

The bottom pane shows the output of the server in the Run console. The output consists of multiple lines of data, each representing a received message from the client. The data is formatted as a string of hexadecimal values, such as `b'39.900001525878906,34.900001525878906'`.

## CLIENT Sending Data:



```
Thonny - /home/p/dht22/Npiotimp1.py @ 7:18
File Edit View Run Tools Help
Npiotimp1.py x npteltemp.py x npteltemp2.py x
1 import socket
2 import sys
3 import RPi.GPIO as GPIO
4 from time import sleep
5 import Adafruit_DHT
6
7 def sensordata():|
8     GPIO.setmode(GPIO.BCM)
9     GPIO.setwarnings(False)
10    Sensor=Adafruit_DHT.DHT22
11    humidity, temperature = Adafruit_DHT.read_retry(Sensor,4)
12    return(humidity, temperature)
13
Shell x
sending "40.099998474121094,34.700000076293945"
sending "40.099998474121094,34.700000076293945"
sending "40.099998474121094,34.700000076293945"
sending "40.099998474121094,34.700000076293945"
sending "40.099998474121094,34.700000076293945"
```

## RESULT :

### Viva

1. Which is connection less protocol?  
Ans:
2. What is meant by reliable communication? Which is reliable?  
Ans:
3. TCP/UDP protocol comes under which layer?  
Ans:
4. In TCP/UDP Which is faster and why?  
Ans:
5. What are the advantages of TCP?  
Ans:

## **Experiment 12: To upload/update Temperature and Humidity data from ThingSpeak cloud based IOT dash board with Raspberry Pi**

**AIM:** To Interface & write a python program to upload/update temperature and humidity data from ThingSpeak cloud based IOT dashboard.

### **Components required:**

- Raspberry Pi 3b+ board
- 1 HDMI Probe
- Keyboard
- Mouse
- Monitor
- SD Card with NOOBS OS
- DHT22

### **DHT22:**

**Temperature:** A measure of the warmth or coldness of an object or substance with reference to some standard value.

**Humidity:** A quantity representing the amount of water vapour in the atmosphere.

### **DHT22: Digital Humidity Temperature sensor**

- The DHT-22 (also named as AM2302) is a digital-output relative humidity and temperature sensor. It uses a capacitive humidity sensor and a thermistor to measure the surrounding air, and spits out a digital signal on the data pin.

### **Features**

- The range of operating voltage() is 3V to 5V power.
- It measures temperature in a range of -40°C to +125°C with an accuracy of  $\pm 0.5$  degrees.
- The measuring range of humidity is from 0 to 100% with an accuracy of 2-5%.
- The maximum operating current for DHT22 sensor is 2.5mA.
- The sampling rate for DHT22 sensor is 0.5 Hz. It takes measurement once every 2 seconds.

### Pin Description:

The DHT22 sensor is very simple and easy to use. It has only four pins.

1. **Vcc** is the power pin. Apply voltage in a range of 3.5V to 5.0V at this pin.
2. **DataOut** is the digital output pin. It sends out the value of measured temperature and humidity in the form of serial data.
3. **N/C** is a not connect pin.
4. **GND** :Connect the GND pin to main ground.

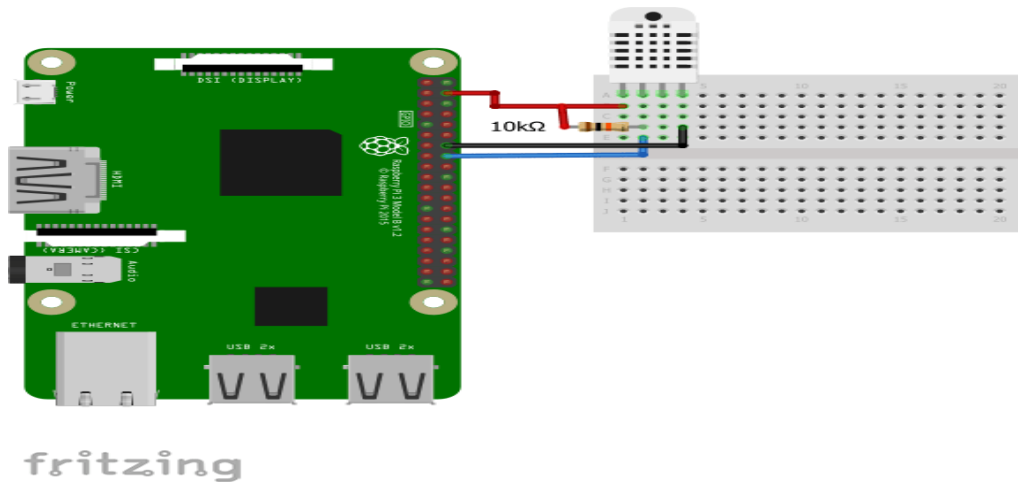
**Figure:Pin Configuration of DHT22**



### ThingSpeak:

- ThingSpeak is an open-source Internet of Things application and API to store and retrieve data from things using the HTTP and MQTT protocol over the Internet or via a Local Area Network.
- ThingSpeak is an IoT analytics platform service that allows you to aggregate, visualize and analyze live data streams in the cloud. You can send data to ThingSpeak from your devices, create instant visualization of live data and send alerts.

### Circuit Diagram:



### Program:

```
import sys
import urllib.request
from time import sleep
import Adafruit_DHT as dht

#EnterYour APIkey here myAPI='T736KR658LSN9USN'
#URL where we will send the data ,Don't change it
baseURL='https://api.thingspeak.com/update?api_key=%s'
%myAPI
defDHT22_data():
    # Reading from DHT22 and storing the temperature and humidity
    humi,temp= dht.read_retry(dht.DHT22,4)
    return humi,temp
whileTrue:
    try:
        humi, temp = DHT22_data()

        #IfReading is valid
        if isinstance(humi,float)and isinstance(temp,float):# Formatting to two
            decimal places
            humi= '%.2f' %humi
            temp= '%.2f' %temp print('temperature:',temp)
            #Sending the data to thingspeak
            conn = urllib.request.urlopen(baseURL + '&field1=%s&field2=%s' %(temp,
            humi))
```

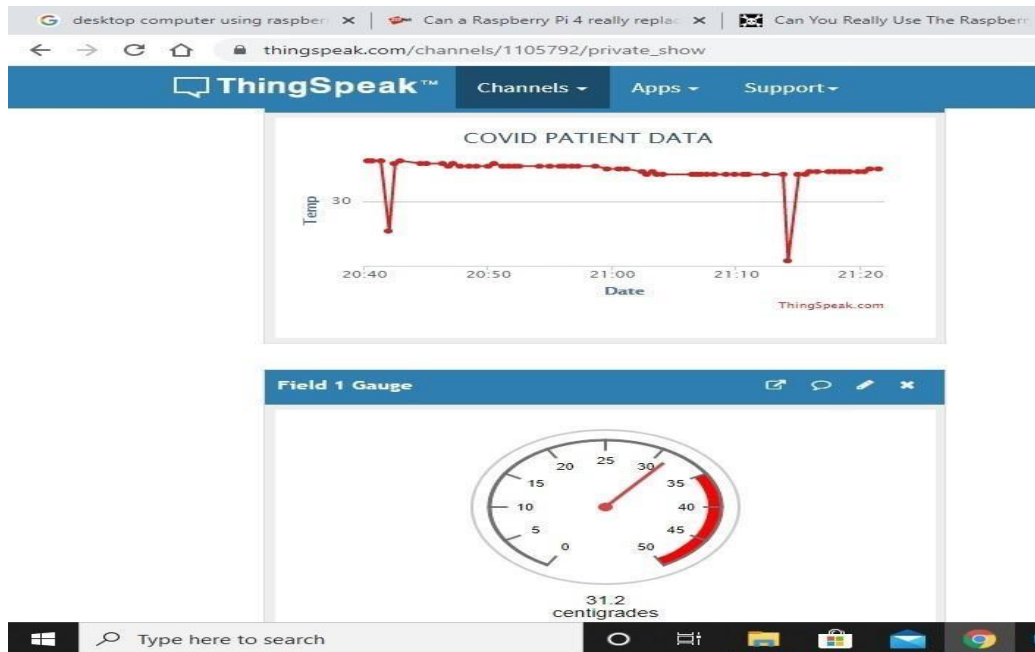
```

        res=conn.read()
        print(res)
        # Closing the connection
        conn.close()
    else:
        print('Error')
        #DHT22 requires 2 seconds to give a reading,so make sure
        #to add delay of above 2 seconds.
        sleep(20)
except: break

```

### Output:

Temperature Monitoring using IOT Dashboard



### Result:

### Viva

1. What is Thingspeak?

Ans:

2. What is the importance of  
urllib.request module?Ans:
3. Which library is used to  
work with DHT22?Ans:
4. What is API?  
Ans: